

Kinetic-Optimal Scheduling with Moment Correction for Metric-Induced Discrete Flow Matching in Zero-Shot Text-to-Speech

Dong Yang^{1*}, Yiyi Cai², Haoyu Zhang¹, Yuki Saito¹, Hiroshi Saruwatari¹

¹The University of Tokyo, ²Independent Researcher

*ydqmkxx@gmail.com

Abstract

Metric-induced discrete flow matching (MI-DFM) exploits token-latent geometry for discrete generation, but its practical use is limited by two issues: heuristic schedulers requiring hyperparameter search, and finite-step path-tracking error from its first-order continuous-time Markov chain (CTMC) solver. We address both issues. First, we derive a kinetic-optimal scheduler for prescribed scalar-parameterized probability paths, and instantiate it for MI-DFM as a training-free numerical schedule that traverses the path at constant Fisher–Rao speed. Second, we introduce a finite-step moment correction that adjusts the jump probability while preserving the CTMC jump destination distribution. We validate the resulting method, GibbsTTS, on codec-based zero-shot text-to-speech (TTS). Under controlled comparisons with a unified architecture and large-scale dataset, GibbsTTS achieves the best objective naturalness and is preferred in subjective evaluations over masked discrete generative baselines. Additionally, in comparison with the evaluated state-of-the-art TTS systems, GibbsTTS shows strong speaker similarity, achieving the highest similarity on three of four test sets and ranking second on the fourth.

Project page: <https://ydqmkxx.github.io/GibbsTTSPROJECT>

1 Introduction

Metric-induced discrete flow matching (MI-DFM) [22] provides a principled way to use the geometry of discrete tokens in generative modeling. Unlike mask-source discrete generative models that corrupt data by replacing tokens with a special mask state, MI-DFM starts from a uniform token distribution and defines intermediate distributions as Gibbs distributions over token-latent distances, which gradually concentrate on the target token. Although it has been applied to several tasks [29, 7, 33, 17], its broader application remains limited by two issues. First, its time scheduler is typically heuristic and tuned through hyperparameter search. This scheduler controls how fast the Gibbs distribution concentrates toward the target token, and therefore has a strong impact on sampling quality. Second, MI-DFM inference relies on a continuous-time Markov chain (CTMC) solver derived from infinitesimal dynamics. In practice, it is applied with a finite number of inference steps, where the standard first-order solver can incur significant path-tracking error.

This work first addresses the scheduling problem from a Fisher–Rao geometric perspective. Prior work [22] derives kinetic-optimal velocities for arbitrary discrete probability paths, and mask-source mixture paths [11, 24] admit closed-form kinetic-optimal schedules. However, such closed-form schedules do not generally exist for arbitrary discrete probability paths. We therefore study kinetic-optimal time scheduling for prescribed scalar-parameterized paths $p(x; \kappa)$. We show that the kinetic-optimal scheduler traverses the path at constant Fisher–Rao speed. For MI-DFM, the Fisher information depends on the full token-distance distribution, so we construct the scheduler numerically from distance matrices computed from token embeddings. This gives a training-free scheduler and avoids heuristic scheduler families or downstream hyperparameter search.

We further introduce a finite-step moment correction for CTMC sampling. The correction preserves the CTMC jump destination distribution and adjusts only the jump probability, so that the one-step update better matches a reference moment at the next time step. For MI-DFM, we use a local Fisher–Rao tangent statistic, yielding a lightweight correction that improves finite-step path tracking, with consistent empirical gains shown in Section 7.

We validate the resulting method on codec-based [6] zero-shot text-to-speech (TTS), where discrete flow matching remains less explored. Codec-based TTS is a natural application for MI-DFM, since acoustic tokens are associated with learned codebook embeddings and nearby tokens often correspond to acoustically similar representations. To evaluate the algorithmic effect, we build a DiT-based [19] codec-token TTS model and conduct controlled comparisons under a unified architecture and large-scale dataset. Experiments on English and Chinese zero-shot TTS show that our method achieves the best objective naturalness against masked discrete generative baselines, and is further preferred in subjective evaluations. Our contributions are summarized as follows:

- We formulate kinetic-optimal time scheduling for prescribed scalar-parameterized discrete probability paths, complementing prior work [22] on kinetic-optimal velocities. We instantiate this framework for MI-DFM and derive a numerical scheduler from token-latent distance matrices. This avoids heuristic scheduler families and downstream hyperparameter search, which is a limitation identified in [22].
- We introduce a finite-step moment correction for CTMC sampling that reduces path-tracking error by adjusting the jump probability while preserving the CTMC jump destination distribution.
- We build a DiT-based codec-token TTS model and conduct controlled comparisons under a unified architecture and large-scale dataset, providing a fair evaluation of MI-DFM against masked discrete generative baselines. To the best of our knowledge, this is the first study of MI-DFM for TTS.
- Experiments validate the proposed methods. Comparisons against state-of-the-art zero-shot TTS systems further indicate the potential of MI-DFM for TTS and support the proposed model as a credible testbed for evaluating discrete generative algorithms.

For clarity in the following sections and to facilitate future comparisons, we refer to the full model equipped with the kinetic-optimal scheduler and finite-step moment correction as *GibbsTTS*, reflecting the Gibbs-distribution form of the metric-induced path.

2 Preliminaries

2.1 Discrete flow matching

Probability paths. We denote a discrete token sequence $x = (x^1, x^2, \dots, x^N) \in \mathcal{S} = [s]^N$, $[s] = \{1, \dots, s\}$, where N is the length of the sequence, s is the vocabulary size of the tokens, and $\mathcal{S}$ is the set of possible sequences. Let $p(x)$ and $q(x)$ denote the source and target *probability mass functions* (PMFs) over the space $\mathcal{S}$, the goal of discrete flow matching is to learn the transformation from $p(x)$ to $q(x)$. We consider time-dependent *probability paths* $p_t(x)$, $t \in [0, 1]$ with the form

$$p_t(x) = \sum_{x_1 \in \mathcal{S}} p_t(x|x_1)q(x_1), \quad p_t(x|x_1) = \prod_{i=1}^N p_t(x^i|x_1^i) \quad (1)$$

where $p_t(x^i|x_1^i)$ is a conditional probability path, and the boundary conditions are $p_0(x^i|x_1^i) = p_0(x^i)$ and $p_1(x^i|x_1^i) = \delta_{x_1^i}(x^i)$. Here, δ_y denotes the delta function on $\mathcal{S}$, defined by

$$\delta_y(x) = \prod_{i=1}^N \delta_{y^i}(x^i), \quad \text{where } \delta_{y^i}(x^i) = \begin{cases} 1 & x^i = y^i \\ 0 & x^i \neq y^i \end{cases}. \quad (2)$$

The mixture paths [11] with x_1^i -dependent schedulers introduced in [24] are defined as

$$p_t(x^i|x_1^i) = (1 - \kappa_t(x_1^i))p(x^i) + \kappa_t(x_1^i)\delta_{x_1^i}(x^i), \quad \kappa_0(\cdot) = 0 \text{ and } \kappa_1(\cdot) = 1. \quad (3)$$

Probability velocities. We consider a continuous-time Markov chain (CTMC) $X_t \sim p_t$, $t \in [0, 1]$ in $\mathcal{S}$, and each token in X_t is updated independently with the time step $h > 0$:

$$\mathbb{P}(X_{t+h} = x | X_t = z) = \delta_z(x) + hu_t(x, z) + o(h), \quad u_t(x, z) = \sum_{i=1}^N u_t^i(x^i, z) \prod_{j \neq i} \delta_{z^j}(x^j), \quad (4)$$

where $o(h)$ denotes a higher-order infinitesimal and is omitted in the following derivation. u_t is called the *probability velocity*. When $x^i \neq z^i$, $u_t^i(x^i, z) \geq 0$ and can be expressed as a marginal form:

$$u_t^i(x^i, z) = \frac{\kappa_t}{1-\kappa_t} [p_{1|t}^i(x^i|z) - \delta_{z^i}(x^i)], \quad p_{1|t}^i(x^i|z) = \sum_{x_1 \in \mathcal{S}} \delta_{x_1^i}(x^i) \frac{p_t(z|x_1)q(x_1)}{p_t(z)}, \quad (5)$$

where $p_{1|t}(x^i|z)$ is the posterior probability. That is, $u_t^i(x^i, z) = \mathbb{E}_{x_1 \sim p_{1|t}(\cdot|z)}[u_t^i(x^i, z^i|x_1^i)]$.

According to the rate condition, when $x^i = z^i$,

$$u_t^i(z^i, z^i|x_1^i) = - \sum_{x^i \neq z^i} u_t^i(x^i, z^i|x_1^i). \quad (6)$$

Training. A simple training objective of DFM is

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_1 \sim q(\cdot), x \sim p_t(\cdot|x_1)} \sum_{i=1}^N [-\log p_{1|t}^{\theta,i}(x_1^i|x)]. \quad (7)$$

The learnable parameters θ are trained to predict the target states from the states sampled along the probability paths.

Inference. At inference, [22] proposes the following sampling process that avoids the summation computation in Eq. 5:

$$\hat{x}_1^i \sim p_{1|t}^{\theta,i}(\cdot|x_t), \quad \hat{x}_{t+h}^i \sim \delta_{x_t^i}(\cdot) + hu_t^i(\cdot, x_t^i|\hat{x}_1^i) \quad (8)$$

2.2 Kinetic-optimal velocities and probability paths

Notation. From here on, we use a simplified notation following [22], where the state space is now $[s]$ and $x, z \in [s]$. The position superscript i is omitted for clarity.

Kinetic-optimal u_t^* . [22] proposes a general formulation to construct a kinetic-optimal velocity $u_t^*(x)$ for an arbitrary discrete probability path $p_t(x)$, from the perspective of minimizing the symmetric kinetic energy:

$$u_t^*(x, z) = \begin{cases} \frac{[p_t(z)\dot{p}_t(x) - \dot{p}_t(z)p_t(x)]_+}{p_t(z)}, & \text{if } x \neq z \text{ and } p_t(z) > 0, \\ 0, & \text{if } x \neq z \text{ and } p_t(z) = 0. \end{cases} \quad (9)$$

The diagonal where $x = z$ is then determined by the rate condition in Eq. 6.

Kinetic optimal p_t^* . [22] formulates the problem of finding the kinetic-optimal probability path by introducing $a_t(x) = \sqrt{p_t(x)}$:

$$\min_a \int_0^1 \sum_x \dot{a}_t(x)^2 dt, \quad \text{s.t.} \sum_x a_t(x)^2 = 1, \quad \forall t \in [0, 1], \quad a_0(x) = \sqrt{p(x)}, \quad a_1(x) = \sqrt{q(x)}, \quad (10)$$

The solution is the geodesic on the probability hypersphere, and the optimal probability path is $p_t^*(x) = a_t(x)^2$. When $q(x) = \delta_{x_1}(x)$, this path takes the mixture form in Eq. 3, where

$$\kappa_t(x_1) = 1 - \frac{\sin^2((1-t)\Omega(x_1))}{\sin^2 \Omega(x_1)}, \quad \text{where } \Omega(x_1) = \arccos \sqrt{p(x_1)}. \quad (11)$$

When it is used with a mask source distribution that $p(x) = \delta_{\text{mask}}(x)$, then $p(x_1) = 0$ and $\Omega(x_1) = \frac{\pi}{2}$, thus $\kappa_t(x_1) = \sin^2(\frac{\pi}{2}t)$.

2.3 Metric-induced discrete flow matching

[22] further introduces a metric-induced $p_t(x)$ in Eq 12, which we call metric-induced DFM (MI-DFM). It exploits the geometric structure of the tokenizer latent space with the form:

$$p_t(x|x_1) = \text{softmax}(-\beta_t d(x, x_1)), \quad (12)$$

where $\beta : [0, 1] \rightarrow \mathbb{R}_{\geq 0}$ is a monotonic scheduler with $\beta_0 = 0$, $\beta_1 = \infty$, and $d : [s] \times [s] \rightarrow \mathbb{R}_{\geq 0}$ such that $d(x, x_1) = 0 \Leftrightarrow x = x_1$. This p_t takes the form of a Gibbs distribution, where the energy of each token is determined by its distance to the target token x_1 in the tokenizer latent space. At $t = 0$, all tokens have equal sampling probability, corresponding to random token initialization, whereas as $t \rightarrow 1$, the distribution collapses to $\delta_{x_1}(x)$. Combining Eq. 9 with Eq. 12, the u_t^* is given by

$$u_t^*(x, z|x_1) = p_t(x|x_1) \dot{\beta}_t [d(z, x_1) - d(x, x_1)]_+. \quad (13)$$

The distance function $d(\cdot, \cdot)$ can be instantiated using any valid metric. A direct choice is the ℓ_p -norm distance between the latent embeddings $\mathbf{e}(\cdot)$ associated with tokens, where β_t is in a heuristic form, and lp, c, a are hyperparameters:

$$d(x, x_1) = \|\mathbf{e}(x) - \mathbf{e}(x_1)\|_2^{\text{lp}}, \quad \beta_t = c \left(\frac{t}{1-t} \right)^a. \quad (14)$$

3 Kinetic-optimal time scheduling

In this section, we study the kinetic-optimal time scheduling problem for a prescribed probability path. We consider a fixed one-dimensional family of categorical distributions $p(x; \kappa)$, where $\kappa \in [0, \kappa_{\max}]$ is a scalar path parameter that specifies a geometric curve on the probability simplex. A monotonic time scheduler κ_t induces the time-dependent probability path $p_t(x) = p(x; \kappa_t)$, $t \in [0, 1]$. Here, we do not optimize over all possible probability paths. Instead, the geometric curve $p(x; \kappa)$ is fixed, and our goal is to find the kinetic optimal κ_t^* that minimizes the Fisher–Rao path energy: $\kappa_t^* = \arg \min_{\kappa_t \in \mathcal{K}} \mathcal{E}_{FR}[\kappa_t]$, $\mathcal{K} = \{\kappa_t : \kappa_0 = 0, \kappa_1 = \kappa_{\max}, \dot{\kappa}_t \geq 0\}$. Under this formulation, changing κ_t only reparameterizes the same geometric path. Therefore, the Fisher-Rao length is invariant to the scheduler, while the Fisher-Rao energy depends on the speed along the path. The kinetic-optimal scheduler is obtained by enforcing a constant Fisher–Rao speed.

To measure the cost of the probability path in the discrete state space, we utilize the path energy induced by the Fisher-Rao metric [14]. For a categorical distribution, the squared Riemannian velocity of the probability path p_t is given by $\|\dot{p}_t\|_{g_{FR}}^2 = \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}$. Therefore, the Fisher-Rao path energy over $t \in [0, 1]$ can be explicitly formulated as $\mathcal{E}_{FR}[p] = \int_0^1 \|\dot{p}_t\|_{g_{FR}}^2 dt = \int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)} dt$

This energy is consistent with the kinetic energy defined in Eq. 10. Specifically, with $a_t(x) = \sqrt{p_t(x)}$, we have $\dot{a}_t(x) = \frac{\dot{p}_t(x)}{2\sqrt{p_t(x)}}$, $\mathcal{E}_{FR}[p] = 4 \int_0^1 \sum_x \dot{a}_t(x)^2 dt$. Therefore, the kinetic energy minimization problem defined in Eq. 10 is equivalent to minimizing the Fisher-Rao path energy. In our setting, the geometric path $p(x; \kappa)$ is prescribed, and we only optimize its time reparameterization κ_t .

3.1 Kinetic-optimal scheduling via Fisher-Rao geometry

We derive the kinetic-optimal scheduler for a prescribed geometric path $p(x; \kappa)$. The following lemmas and proposition show that the solution is obtained by reparameterizing the path with constant Fisher–Rao speed, which leads to a constructive scheduler determined by the Fisher information.

The proofs of Lemma 1, Lemma 2, and Proposition 1 are provided in Appendix B.

Lemma 1 (Scheduler-invariant Fisher-Rao length). *Let $p_t(x) = p(x; \kappa_t)$, where $p(x; \kappa)$ is a differentiable probability path with $\kappa \in [0, \kappa_{\max}]$, and κ_t is a monotonic scheduler with $\kappa_0 = 0$ and $\kappa_1 = \kappa_{\max}$. Then the Fisher-Rao length is invariant to the choice of κ_t :*

$$\ell[\kappa_t] = \int_0^1 \|\dot{p}_t\|_{g_{FR}} dt = \int_0^{\kappa_{\max}} \sqrt{\mathcal{I}(\kappa)} d\kappa \triangleq L, \quad (15)$$

where $\mathcal{I}(\kappa) = \sum_x \frac{(\partial_{\kappa} p(x; \kappa))^2}{p(x; \kappa)}$ is the Fisher information of the path parameter κ .

Lemma 2 (Constant-speed characterization). *For a prescribed probability path $p(x; \kappa)$, let $p_t^*(x) = p(x; \kappa_t^*)$ be the path induced by the kinetic-optimal scheduler κ_t^* . Then κ_t^* minimizes $\mathcal{E}_{FR}[\kappa_t]$ over $\kappa_t \in \mathcal{K}$ if and only if it satisfies*

$$\|p_t^*\|_{g_{FR}} = L, \quad \forall t \in [0, 1], \quad (16)$$

where L is the scheduler-invariant Fisher-Rao length defined in Lemma 1.

Proposition 1 (Kinetic-optimal time scheduler). *Let $p_t(x) = p(x; \kappa_t)$ be a prescribed path parameterized by a monotonic scheduler κ_t , with $\kappa_0 = 0$ and $\kappa_1 = \kappa_{\max}$. Assume $\mathcal{I}(\kappa) > 0$ on $[0, \kappa_{\max}]$. The kinetic-optimal scheduler and its time derivative are given by:*

$$\kappa_t^* = F^{-1}(t), \quad \dot{\kappa}_t^* = \frac{L}{\sqrt{\mathcal{I}(\kappa_t^*)}}, \quad \text{where } F(\kappa) = \frac{\int_0^\kappa \sqrt{\mathcal{I}(\xi)} d\xi}{L}.$$

The mixture path in Eq. 3 already admits a closed-form kinetic-optimal scheduler (Section 2.2; [22]), and its mask-source form has also been shown to be Fisher–Rao optimal for masked discrete diffusion models [39]. Since this case is already well understood, we focus the main text on metric-induced paths and provide the mixture-path derivation in Appendix D as a consistency check.

3.2 Numerical construction for metric-induced paths

We now instantiate Proposition 1 for the metric-induced path defined in Eq. 12. We consider it as $p(x \mid x_1; \beta) = \text{softmax}(-\beta d(x, x_1))$, where the scheduler given by $\kappa_t = \beta_t$.

For a fixed x_1 , the Fisher information as defined in Lemma 1 with respect to β is $\mathcal{I}_{x_1}(\beta) = \mathbb{E}_{x \sim p(\cdot \mid x_1; \beta)} \left[(\partial_\beta \log p(x \mid x_1; \beta))^2 \right]$. Then, we obtain

$$\partial_\beta \log p(x \mid x_1; \beta) = \mathbb{E}_{y \sim p(\cdot \mid x_1; \beta)} [d(y, x_1)] - d(x, x_1), \quad \mathcal{I}_{x_1}(\beta) = \text{Var}_{x \sim p(\cdot \mid x_1; \beta)} [d(x, x_1)]. \quad (17)$$

Thus, for the metric-induced $p_t(x)$ in a Gibbs distribution form, the Fisher information $\mathcal{I}_{x_1}(\beta)$ is exactly the variance of the distance to the target token. Although $\mathcal{I}_{x_1}(\beta)$ admits a simple variance form, it still depends on the full set of distances $\{d(x, x_1)\}_{x \in [s]}$ and the induced distribution $p(x \mid x_1; \beta)$, and therefore does not generally admit a simple analytic expression.

We therefore construct the scheduler numerically. Since the limiting endpoint corresponds to $\beta \rightarrow \infty$, we first choose a sufficiently large $\beta_{\max}$ to approximate this limit, as detailed in Algorithm 1.

We then construct a one-dimensional inverse-temperature grid $\{\beta_i\}_{i=1}^I$, where $0 = \beta_1 < \dots < \beta_I = \beta_{\max}$, and evaluate the Fisher information on this grid. The resulting values are used to compute the cumulative Fisher–Rao arc length and invert it on a uniform time grid $\{t_j\}_{j=1}^T$. This yields the lookup tables $\{\beta_j^*\}_{j=1}^T$ and $\{\dot{\beta}_j^*\}_{j=1}^T$, as detailed in Algorithm 2. During training and inference, β_t^* and $\dot{\beta}_t^*$ for arbitrary $t \in [0, 1]$ are obtained by linear interpolation from these tables. This method does not require model training to search for scheduler hyperparameters. The remaining choices are numerical construction parameters, where $\beta_{\max}$ is automatically determined by Algorithm 1, while the others control the numerical resolution. Increasing the grid resolution improves the approximation accuracy but increases precomputation and storage costs, so they are selected based on an accuracy-cost trade-off rather than downstream experimental search.

Please see more details in Appendix C.

4 Finite-step moment correction

The kinetic-optimal velocity in Eq. 9 and its metric-induced form in Eq. 13 define infinitesimal CTMC dynamics. In practice, inference uses a finite number of steps, and the standard first-order solver approximates the evolution over $[t, t+h]$ using the instantaneous rate at time t . The resulting one-step transition is exact only in the infinitesimal-step limit, and may incur non-negligible path-tracking error when the number of inference steps is limited.

We introduce a finite-step moment-matched correction to reduce this practical discretization error. The correction is not intended as a higher-order CTMC integrator or an exact finite-step transition kernel. Instead, it keeps the normalized instantaneous CTMC jump destination distribution unchanged

and adjusts only the probability of accepting a jump, so that a chosen scalar moment better matches a reference value at time $t + h$. We first describe the correction as a generic framework parameterized by the choice of moment statistic and reference moment, and then instantiate it for metric-induced DFM. As a consistency check, Appendix F shows that the generic correction reduces to the known exact finite-step update for the standard mixture path when using the target-state indicator moment and the corresponding state-conditional reference moment.

4.1 A generic moment-corrected CTMC step

We use the single-token notation from Section 2. At inference, given the current token z at time t , we first sample a predicted endpoint token $\hat{x}_1 \sim p_{1|t}^\theta(\cdot | z)$ from the learned posterior. For the conditional rate $u_t(x, z | \hat{x}_1)$, we define the jump intensity and the jump destination distribution as

$$\lambda_t(z | \hat{x}_1) = \sum_{x \neq z} u_t(x, z | \hat{x}_1), \quad \pi_t(x | z, \hat{x}_1) = \frac{u_t(x, z | \hat{x}_1)}{\lambda_t(z | \hat{x}_1)}, \text{ for } x \neq z. \quad (18)$$

When $\lambda_t(z | \hat{x}_1) = 0$, no jump is performed. The first-order CTMC solver introduced in [22] uses $\rho_{\text{base}} = 1 - \exp(-h\lambda_t(z | \hat{x}_1))$ as the probability of making a jump, and samples the destination from $\pi_t(\cdot | z, \hat{x}_1)$. This solver is exact only in the infinitesimal-step limit. For a finite step size h , the one-step transition may deviate from the reference distribution $p_{t+h}(\cdot | \hat{x}_1)$. We therefore introduce a lightweight finite-step correction that keeps the jump destination distribution π_t fixed, and only adjusts the jump probability. Specifically, let $\phi_t(x | \hat{x}_1)$ be a scalar statistic that measures the progress of state x along the path, and let $m_{t+h}(z, \hat{x}_1)$ be the reference moment that the one-step transition should match. If a jump probability $\rho \in [0, 1]$ is used, the post-step moment is

$$(1 - \rho)\phi_t(z | \hat{x}_1) + \rho\bar{\phi}_t(z, \hat{x}_1), \quad \text{where } \bar{\phi}_t(z, \hat{x}_1) = \mathbb{E}_{y \sim \pi_t(\cdot | z, \hat{x}_1)} [\phi_t(y | \hat{x}_1)]. \quad (19)$$

We first obtain the moment-matching jump probability by solving the following one-dimensional least-squares problem:

$$\rho^* = \arg \min_{\rho \in \mathbb{R}} [(1 - \rho)\phi_t(z | \hat{x}_1) + \rho\bar{\phi}_t(z, \hat{x}_1) - m_{t+h}(z, \hat{x}_1)]^2. \quad (20)$$

When $\phi_t(z | \hat{x}_1) \neq \bar{\phi}_t(z, \hat{x}_1)$, the solution to Eq. (20) is

$$\rho^* = \frac{\phi_t(z | \hat{x}_1) - m_{t+h}(z, \hat{x}_1)}{\phi_t(z | \hat{x}_1) - \bar{\phi}_t(z, \hat{x}_1)}. \quad (21)$$

Since ρ^* serves as a jump probability, it must lie in $[0, 1]$. We use the moment-matching correction only when $0 \leq \rho^* \leq 1$, otherwise fall back to the original first-order CTMC solver:

$$\rho_{\text{corr}} = \begin{cases} \rho^*, & \text{if } \phi_t(z | \hat{x}_1) \neq \bar{\phi}_t(z, \hat{x}_1) \text{ and } 0 \leq \rho^* \leq 1, \\ \rho_{\text{base}}, & \text{otherwise.} \end{cases} \quad (22)$$

When $\phi_t(z | \hat{x}_1) = \bar{\phi}_t(z, \hat{x}_1)$, the post-step moment is independent of ρ . In this degenerate case, we also use ρ_{base} . The statistic ϕ_t and the reference moment m_{t+h} should be chosen according to the structure of each probability path.

4.2 Application to metric-induced paths

For the metric-induced path in Eq. 12, given a predicted $\hat{x}_1$, we have $p_t(x | \hat{x}_1) = \text{softmax}(-\beta_t d(x, \hat{x}_1))$. Since the purpose of the correction is to reduce the finite-step deviation from the reference path, we use the local Fisher-Rao tangent statistic:

$$\phi_t(x | \hat{x}_1) = \partial_t \log p_t(x | \hat{x}_1) = \dot{\beta}_t (\mathbb{E}_{y \sim p_t(\cdot | \hat{x}_1)} [d(y, \hat{x}_1)] - d(x, \hat{x}_1)). \quad (23)$$

The corresponding reference moment is taken under the next-time distribution:

$$m_{t+h}(z, \hat{x}_1) = \mathbb{E}_{y \sim p_{t+h}(\cdot | \hat{x}_1)} [\phi_t(y | \hat{x}_1)] = \dot{\beta}_t (\mathbb{E}_{y \sim p_t(\cdot | \hat{x}_1)} [d(y, \hat{x}_1)] - \mathbb{E}_{y \sim p_{t+h}(\cdot | \hat{x}_1)} [d(y, \hat{x}_1)]). \quad (24)$$

Since $\mathbb{E}_{y \sim p_t(\cdot | \hat{x}_1)} [\partial_t \log p_t(y | \hat{x}_1)] = 0$, this reference moment measures the displacement from p_t to p_{t+h} after projection onto the local Fisher-Rao tangent direction. Note that ϕ_t is the tangent

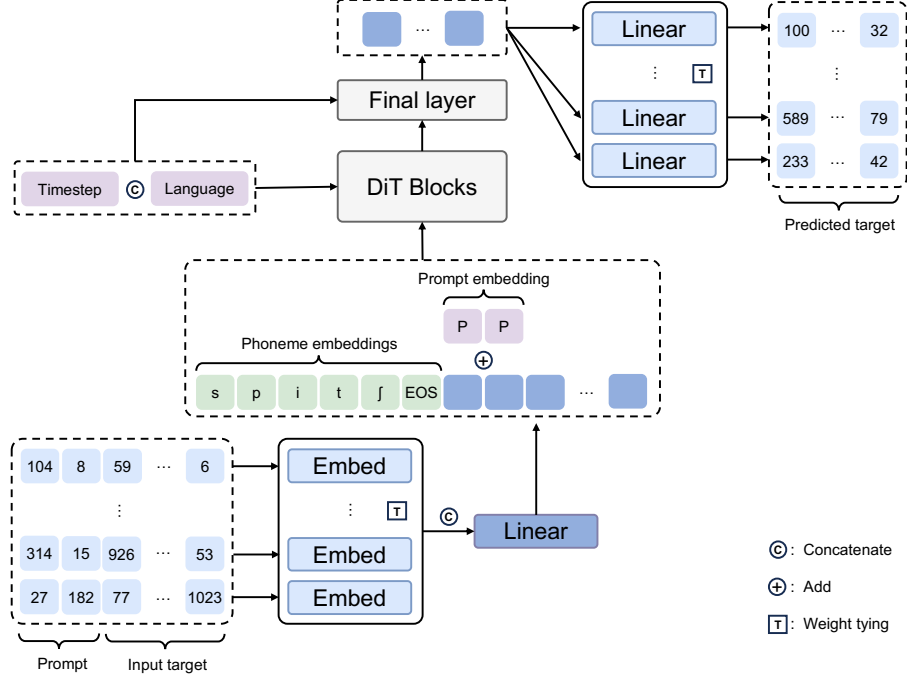


Figure 1: Architecture of the proposed model.

statistic at time t , so evaluating it under p_{t+h} is itself a first-order approximation in h . Matching this moment thus aligns the one-step update with the reference path along this tangent direction, while preserving the CTMC jump destination distribution. Then, according to Eq. 19, we have

$$\bar{\phi}_t(z, \hat{x}_1) = \mathbb{E}_{y \sim \pi_t(\cdot | z, \hat{x}_1)} [\phi_t(y | \hat{x}_1)] = \dot{\beta}_t \left(\mathbb{E}_{y \sim p_t(\cdot | \hat{x}_1)} [d(y, \hat{x}_1)] - \mathbb{E}_{y \sim \pi_t(\cdot | z, \hat{x}_1)} [d(y, \hat{x}_1)] \right). \quad (25)$$

Substituting Eqs. 23, 24, and 25 into Eq. 21 yields

$$\rho^* = \frac{d(z, \hat{x}_1) - \mathbb{E}_{y \sim p_{t+h}(\cdot | \hat{x}_1)} [d(y, \hat{x}_1)]}{d(z, \hat{x}_1) - \mathbb{E}_{y \sim \pi_t(\cdot | z, \hat{x}_1)} [d(y, \hat{x}_1)]}. \quad (26)$$

The final jump probability is obtained from Eq. 22. Since $\dot{\beta}_t$ cancels, the corrected jump probability depends only on expected distances and does not recover the infinitesimal CTMC limit as $h \rightarrow 0$. Therefore, the correction should be interpreted as a finite-step approximation aimed at reducing discretization error at practical NFEs.

5 Model

For conciseness, this section summarizes the main model design, while detailed training and inference procedures are provided in Appendix E. Specifically, Algorithms 3 and 4 describe the training and inference processes, respectively.

Backbone. As shown in Fig. 1, we adopt a DiT [19] backbone and leverage RoPE position embedding [26], SwiGLU [23], and RMSNorm [38]. The timestep and language embeddings are concatenated and used as conditioning for the adaLN-Zero layers in DiT.

Input construction. We improve the StableTTS text frontend [1] for text normalization and grapheme-to-phoneme conversion. Codec-token embeddings from all RVQ codebooks are concatenated along the channel dimension and linearly projected to a per-frame embedding, which is then concatenated along the time dimension with the phoneme embeddings.

Prompt construction. During training, each utterance is randomly split at a ratio sampled from $U(0, 0.3)$, with the prefix used as the speech prompt and the remainder as the prediction target. A learnable prompt embedding is added to the prompt positions.

Table 1: Model and training configurations.

Model	Size	Dim.	Layers	Dataset	Max. length	Max. batch hours	GPUs	Train time
Base	178M	768	12	Emilia-en	1,024 tokens / 20.48 s	2.912	8	33 hours
Large	399M	1,024	16	Emilia-en/zh	1,536 tokens / 30.72 s	4.369	32	46 hours

Duration predictor. We adopt a rule-based duration predictor inspired by the released implementation of MaskGCT and further refine it for our model. Details are provided in Appendix J.

Codebook-wise loss weighting. We perform full-codebook training and inference, and introduce codebook-wise loss weights in the training objective, as shown in Lines 12–13 of Algorithm 3. We discuss its effect and compare it with the per-layer training strategy of MaskGCT in Appendix I.

6 Experimental setup

6.1 Model implementations

Variants. As shown in Table 1, we set both Base and Large variants. The Base variants are used to reduce computational cost and are employed for hyperparameter search and ablation studies.

Codec. We use the pre-trained acoustic codec released by MaskGCT. The encoder follows DAC [15] and the decoder follows Vocos [25]. It has 12 RVQ layers, a codebook size of 1024, and an embedding dimension of 8. With a 24,000 Hz sampling rate and a hop size of 480, 50 codec frames correspond to 1 second of speech.

Codebook distance matrices. Since the codec uses ℓ_2 -normalized codebook embeddings, we compute the codebook distance as the squared Euclidean distance between these normalized embeddings. The distance matrices in Appendices C and E are instantiated as $\{\mathbf{D}_c\}_{c=1}^C$, where $\mathbf{D}_c$ contains the pairwise squared Euclidean distances between the embeddings of the c -th RVQ codebook.

Masked discrete generative baselines. As discussed in Section 2.1, masked DFM and masked discrete diffusion (DD) share the same training objective. Therefore, for masked models, we evaluate both the standard DFM [11, 18] and the DD-style inference procedure used in MaskGCT. As shown in Table 2, we also compare different time schedulers for masked models: the closed-form kinetic-optimal (KO) scheduler $\kappa_t = \sin^2(\frac{\pi}{2}t)$ discussed in Section 2.2, the scheduler $\kappa_t = t^2$ [18] used in DiFlow-TTS [18], and the scheduler $\kappa_t = \sin(\frac{\pi}{2}t)$ used in MaskGCT [31].

6.2 Training and inference

We train all models for 10 epochs using AdamW [16] without hyperparameter tuning. The peak learning rate is set to 2×10^{-4} , with linear warmup over the first 5% of training steps, followed by cosine decay to 10% of the peak learning rate. We maintain an exponential moving average (EMA) of the model weights with a decay rate of 0.9999. Following MaskGCT, we apply classifier-free guidance (CFG) with a condition drop rate of 0.15, a CFG scale of 2.5, and a rescale factor of 0.75. During training, the DiT backbone uses BF16 precision, except for the transformation in RMSNorm layers, which is performed in FP32. All other computations are performed in FP32. All training, inference, and objective evaluations are conducted on NVIDIA H100 GPUs with 96 GB memory.

Dynamic batching. We adopt a dynamic batching strategy. In Table 1, the maximum length denotes the maximum sequence length of acoustic tokens. Given this maximum length, the number of GPUs, and a gradient accumulation factor of 2 for the Base variants, the maximum amount of audio per batch is 2.912 hours for the Base variants and 4.369 hours for the Large variants.

Training dataset. We use the English (en) and Chinese (zh) subsets of Emilia [12] for training. After preprocessing, their total durations are over 46k and 45k hours, respectively.

Inference. We use 32 function evaluations (NFEs) in the main experiments, and report results with 16 and 64 NFEs in Appendix L. For sampling, the temperature is selected separately for each model on the validation set, whose pipeline is described in Appendix K.

6.3 Evaluations

Evaluation datasets. We use the *test-clean* subset of LibriTTS [37] as the validation set, following [35] to construct prompt-target pairs. For testing, we use the *test-en* and *test-zh* of Seed-TTS [2] test sets, and the *en* and *zh* subsets of CosyVoice 3 [8] test sets.

Objective evaluations. We use UTMOS [21] to evaluate naturalness. Following Seed-TTS [2], we use Whisper-large-v3 [20] to compute word error rate (WER) for English and Paraformer-zh [10] to compute character error rate (CER) for Chinese. For speaker similarity (SIM), we extract speaker embeddings using the WavLM-large [4] speaker verification model and compute the cosine similarity.

Subjective evaluation. We conducted comparative MOS (CMOS, $[-3, 3]$) and similarity MOS (SMOS, $[1, 5]$) tests to evaluate naturalness and similarity, respectively. In each test, we recruited 20 native English or Chinese listeners on Prolific¹ with £9 per hour.

Evaluation protocol. In our experiments, we observe that under a fixed framework, where the model backbone, training procedure, and inference pipeline are kept identical, it is uncommon for one model to outperform another across all objective metrics. Therefore, for hyperparameter selection based on objective evaluation, we consider one model better than another if it achieves better results in at least two of the three metric categories: UTMOS, WER/CER, and SIM.

7 Experimental results and analysis

Table 2 reports the objective results on the Seed-TTS and CosyVoice 3 test sets. *Numerical KO* denotes our proposed numerical kinetic-optimal scheduler, *Grid-searched* denotes the grid-searched heuristic MI-DFM scheduler selected on the validation set (Appendix G), and *w/o corrector* indicates removal of the proposed finite-step moment correction. Under the controlled setting with the same backbone, training procedure, and inference pipeline, GibbsTTS achieves the best overall performance among the evaluated masked discrete generative baselines. On Seed-TTS test sets, GibbsTTS obtains the best results on all three metrics for test-en, and obtains the best UTMOS and SIM on test-zh with CER close to the best baseline. On CosyVoice 3 test sets, it achieves the best UTMOS for both languages and the best English WER, while remaining close to the best results on the other metrics. These results suggest that the metric-induced path is effective for codec-based zero-shot TTS when combined with the proposed kinetic-optimal scheduler and finite-step correction.

The results also show that scheduler choice is both important and decoding-dependent for masked generative baselines. For example, the MaskGCT scheduler $\kappa_t = \sin(\frac{\pi}{2}t)$ performs poorly with masked DFM, but gives the strongest overall performance among masked DD schedulers. Conversely, schedulers that work well for masked DFM do not necessarily work well for masked DD. This suggests that scheduler design is tightly coupled with the discrete generative formulation.

Effect of the kinetic-optimal scheduler. Compared with the grid-searched MI-DFM scheduler, the numerical KO scheduler gives better overall performance across the two test sets. It improves all three metrics on Seed-TTS test-en, and improves UTMOS and SIM on test-zh while maintaining competitive CER. On CosyVoice 3, it brings clear gains over the searched scheduler in most metrics, with a small CER regression on the Chinese subset. Importantly, the numerical KO scheduler does not require downstream hyperparameter search over hand-designed scheduler families. It therefore improves or matches empirical performance while removing a sensitive scheduler-selection procedure.

Effect of the finite-step moment correction. The finite-step moment correction consistently improves MI-DFM across test sets and scheduler choices. Under the numerical KO scheduler, removing the corrector degrades all objective metrics on both Seed-TTS and CosyVoice 3, and the same trend holds for the grid-searched scheduler. The uniform direction and magnitude of the gap across all subset-metric combinations indicate that the gain is not specific to a particular language or scheduler. These results are consistent with the motivation of the correction: a first-order CTMC solver is prone to finite-step path-tracking error, and adjusting the jump probability lets the sampler better follow the reference path while preserving the CTMC jump destination distribution.

Subjective evaluation results. For the listening test, we focus on systems that isolate our two proposed methods, together with the strongest baselines of each family based on the objective evaluation results and the publicly released MaskGCT (*MaskGCT (original)*) for reference. The

¹<https://www.prolific.com>

Table 2: Objective evaluation results. The best value among generated systems is highlighted in **bold**.

(a) Results on Seed-TTS test sets.

Method	Scheduler	test-en			test-zh		
		UTMOS $\uparrow$	WER (%) $\downarrow$	SIM $\uparrow$	UTMOS $\uparrow$	CER (%) $\downarrow$	SIM $\uparrow$
Ground truth	—	3.527	2.020	0.734	2.782	1.327	0.755
Codec reconstructed	—	3.407	2.229	0.695	2.564	1.472	0.725
MI-DFM (GibbsTTS)	Numerical KO	3.651	1.777	0.743	2.712	1.327	0.790
MI-DFM w/o corrector	Numerical KO	3.403	2.120	0.723	2.447	1.777	0.775
MI-DFM	Grid-searched	3.617	1.793	0.729	2.628	1.297	0.784
MI-DFM w/o corrector	Grid-searched	3.380	2.070	0.711	2.381	1.637	0.767
Masked DFM	Closed-form KO	3.639	1.969	0.742	2.656	1.536	0.788
Masked DFM	DiFlow-TTS	3.546	1.827	0.728	2.559	1.308	0.785
Masked DFM	MaskGCT	3.269	2.724	0.712	2.195	3.140	0.762
Masked DD	Closed-form KO	3.634	5.808	0.731	2.706	6.033	0.787
Masked DD	DiFlow-TTS	2.768	9.303	0.672	1.825	10.711	0.734
Masked DD	MaskGCT	3.415	2.338	0.721	2.387	1.583	0.776

(b) Results on CosyVoice 3 test sets. Ground truth targets are not provided in the sets.

Method	Scheduler	en			zh		
		UTMOS $\uparrow$	WER (%) $\downarrow$	SIM $\uparrow$	UTMOS $\uparrow$	CER (%) $\downarrow$	SIM $\uparrow$
MI-DFM (GibbsTTS)	Numerical KO	3.238	4.110	0.691	2.438	4.144	0.780
MI-DFM w/o corrector	Numerical KO	2.850	4.616	0.668	2.135	5.485	0.772
MI-DFM	Grid-searched	3.009	4.506	0.674	2.189	3.706	0.772
MI-DFM w/o corrector	Grid-searched	2.616	4.547	0.653	1.939	4.274	0.755
Masked DFM	Closed-form KO	3.049	5.162	0.695	2.294	4.855	0.781
Masked DFM	DiFlow-TTS	2.925	4.288	0.673	2.141	3.727	0.777
Masked DFM	MaskGCT	2.354	8.767	0.614	1.789	7.235	0.698
Masked DD	Closed-form KO	3.042	18.353	0.677	2.401	14.156	0.776
Masked DD	DiFlow-TTS	1.885	36.133	0.562	1.494	29.180	0.673
Masked DD	MaskGCT	2.657	6.719	0.655	1.903	4.575	0.762

Table 3: Subjective evaluation results. $\ddagger$ indicates $p < 0.05$, and $\dagger$ indicates $p < 0.1$ compared with GibbsTTS. The highest value for each metric is highlighted in **bold**.

Method	Scheduler	en		zh	
		CMOS $\uparrow$	SMOS $\uparrow$	CMOS $\uparrow$	SMOS $\uparrow$
MI-DFM (GibbsTTS)	Numerical KO	0 (ref)	4.18	0 (ref)	4.28
MI-DFM w/o corrector	Numerical KO	-0.362 $\ddagger$	3.86 $\ddagger$	-0.459 $\ddagger$	4.03 $\dagger$
MI-DFM	Grid-searched	-0.257 $\ddagger$	4.05	-0.153	4.21
Masked DFM	Closed-form KO	-0.229 $\dagger$	3.94	-0.224 $\dagger$	4.11
Masked DFM	DiFlow-TTS	-0.286 $\ddagger$	3.91 $\dagger$	-0.247 $\dagger$	4.10
Masked DD	MaskGCT	-0.743 $\ddagger$	3.56 $\ddagger$	-0.647 $\ddagger$	3.87 $\ddagger$
MaskGCT (original)	MaskGCT	-0.762 $\ddagger$	4.06	-0.612 $\ddagger$	4.19

evaluated utterances are sampled in equal proportions from the Seed-TTS and CosyVoice 3 test sets. Since CosyVoice 3 does not provide ground-truth target speech, we cannot include ground-truth samples in the subjective evaluations. Table 3 reports the subjective evaluation results. Since GibbsTTS is used as the reference system in CMOS tests, its CMOS scores are zero. All compared systems obtain negative CMOS scores, indicating that listeners prefer GibbsTTS in naturalness. For speech similarity, GibbsTTS also achieves the highest SMOS on both English and Chinese evaluations. These subjective results support that the proposed scheduler and correction improve perceptual speech quality, rather than only automatic metrics.

Additional analyses are provided in the appendices, including grid search for the heuristic MI-DFM scheduler (Appendix G), shared versus per-codebook scheduler construction (Appendix H), comparison with the per-layer training strategy used in MaskGCT (Appendix I).

8 Comparison with SOTA zero-shot TTS systems

Table 4: Objective evaluation results on Seed-TTS test sets. The best value among generated systems is highlighted in **bold**.

Note: “Emilia” indicates whether Emilia is used for training. INCL. denotes that Emilia is included in the training data.

Model	Size	Emilia	test-en			test-zh		
			UTMOS↑	WER (%) ↓	SIM ↑	UTMOS↑	CER (%) ↓	SIM ↑
Ground truth	—	—	3.527	2.020	0.734	2.782	1.327	0.755
<i>NAR models with the Codec in MaskGCT</i>								
Codec reconstructed	—	—	3.407	2.229	0.695	2.564	1.472	0.725
GibbsTTS (ours)	0.4B	✓	3.651	1.777	0.743	2.712	1.327	0.790
MaskGCT	1.5B	✓	3.582	3.763	0.716	2.647	2.217	0.774
<i>Other NAR models</i>								
F5-TTS [5]	0.3B	✓	3.762	2.020	0.654	2.962	1.741	0.747
OmniVoice [41]	0.6B	×	3.896	1.475	0.741	3.092	0.927	0.778
<i>AR models</i>								
Spark-TTS [30]	0.5B	✓	3.947	2.221	0.572	3.283	1.374	0.658
FireRedTTS-2 [32]	1.5B	×	3.697	2.187	0.664	2.842	1.145	0.728
IndexTTS2 [40]	1.5B	INCL.	3.651	1.735	0.707	2.999	1.112	0.765
CosyVoice 2 [9]	0.5B	×	4.164	2.506	0.656	3.469	1.344	0.752
CosyVoice 3 [8]	0.5B	×	3.949	2.145	0.696	3.325	1.192	0.779
Qwen3-TTS [13]	0.6B	×	4.155	1.711	0.706	3.477	1.129	0.765
Qwen3-TTS	1.7B	×	4.178	1.434	0.712	3.505	1.100	0.770

Table 5: Objective evaluation results on CosyVoice 3 test sets. The best value for each metric is highlighted in **bold**.

Model	Size	Emilia	en			zh		
			UTMOS↑	WER (%) ↓	SIM ↑	UTMOS↑	CER (%) ↓	SIM ↑
<i>NAR models with the Codec in MaskGCT</i>								
GibbsTTS (ours)	0.4B	✓	3.238	4.110	0.691	2.438	4.144	0.780
MaskGCT	1.5B	✓	3.032	7.237	0.690	2.357	6.647	0.773
<i>Other NAR models</i>								
F5-TTS	0.3B	✓	3.188	6.582	0.624	2.337	5.867	0.741
OmniVoice	0.6B	×	3.619	3.182	0.705	2.884	2.735	0.766
<i>AR models</i>								
Spark-TTS	0.5B	✓	3.587	6.840	0.499	2.977	4.802	0.671
IndexTTS2	1.5B	INCL.	3.372	3.987	0.664	2.544	3.064	0.762
CosyVoice 2	0.5B	×	3.850	6.200	0.614	3.042	3.939	0.748
CosyVoice 3	0.5B	×	3.719	4.124	0.676	2.899	3.166	0.776
Qwen3-TTS	0.6B	×	3.680	3.636	0.641	3.221	2.900	0.755
Qwen3-TTS	1.7B	×	3.931	3.406	0.670	3.328	2.653	0.754

Although the primary goal of our experiments is to validate the proposed algorithms under controlled settings, we further compare GibbsTTS with recent SOTA zero-shot TTS systems in Tables 4 and 5. We use their official open-source checkpoints and released inference code.²

These results show that GibbsTTS is not the strongest system in terms of naturalness or WER/CER: larger-scale autoregressive (AR) models and recent NAR systems often achieve higher UTMOS or

²FireRedTTS-2 results on the CosyVoice 3 test sets are not reported. In our runs, its released inference code did not terminate on a subset of these utterances, and imposing a hard iteration cap substantially degraded generation quality.

lower WER/CER. Nevertheless, GibbsTTS achieves the highest speaker similarity on three of the four evaluated test sets. On the remaining CosyVoice 3 English test set, it obtains the second-best similarity score and is only behind OmniVoice, a concurrent NAR zero-shot TTS system. This indicates that MI-DFM is particularly effective at preserving speaker identity in codec-based zero-shot TTS.

A more direct comparison can be made within the group of NAR models using the Codec in MaskGCT. Across both Seed-TTS and CosyVoice 3 test sets, GibbsTTS consistently outperforms MaskGCT on all reported objective metrics, despite using a smaller model size. The subjective evaluation in Table 3 shows the same tendency, with GibbsTTS being preferred in naturalness and obtaining higher similarity ratings than MaskGCT. Moreover, GibbsTTS uses a simpler token representation. MaskGCT involves both semantic tokens and codec tokens, whereas GibbsTTS performs generation directly over codec tokens only. These results suggest that the proposed MI-DFM-based model and overall training-inference framework provide an effective way to model codec tokens directly, without relying on an additional semantic-token stage.

The codec reconstruction results in Table 4 further indicate that codec choice itself affects the UTMOS range of codec-based TTS systems. Even before considering generative modeling errors, the reconstructed speech already shows a noticeable UTMOS gap from the ground-truth speech. When this work was initiated, MaskGCT was still the SOTA codec-based NAR zero-shot TTS system. We therefore conservatively adopt its codec rather than introducing a newer and stronger codec, so that our study focuses more on the comparison of discrete generative modeling strategies.

These comparisons should still be interpreted with caution. The external systems differ in model size, training data, tokenizer, codec, architecture, text frontend, and inference pipeline, so the results are not fully controlled. In particular, GibbsTTS uses a relatively simple G2P-based frontend, whereas some recent systems may benefit from more advanced text normalization. This can affect intelligibility-related metrics such as WER and CER, and therefore the SOTA comparison should not be interpreted as an isolated evaluation of the proposed discrete generative algorithms. The main evidence for the proposed algorithmic contributions comes from the controlled comparisons and ablation studies in the main experiments. The SOTA comparison mainly demonstrates the practical potential of GibbsTTS as a zero-shot TTS system.

9 Limitations and future work

Since the codec used in this work applies ℓ_2 normalization to token embeddings, the distance used in our metric-induced path is equivalent to a cosine distance. Other choices of token distance are not further investigated in this work. Furthermore, after introducing the kinetic-optimal scheduler, the construction of the distance matrix itself remains an important direction for future study. For example, we may further optimize the geometry of token embeddings and make it better suited to MI-DFM.

For an arbitrary discrete probability path, [22] provides the kinetic-optimal velocity formulation, and this work provides a numerical construction for the kinetic-optimal time scheduler. However, other types of probability paths remain unexplored.

In addition, as discussed in Section 4.2, we choose the local Fisher–Rao tangent statistic as the reference moment in our corrector. Other choices of reference moments may lead to better performance and deserve further investigation. Moreover, correctors are also discussed in [22], providing additional insights into possible constructions. Therefore, corrector design, as well as more general decoding strategies for MI-DFM, remains a valuable direction for future work.

Finally, although the proposed algorithms are general, we only evaluate them on zero-shot TTS in this work. Their effectiveness in other domains remains to be explored.

10 Conclusion

We proposed two general algorithmic contributions for discrete flow matching: a kinetic-optimal time scheduler for prescribed scalar-parameterized discrete probability paths, and a finite-step moment correction for practical CTMC sampling that reduces path-tracking error while preserving the jump destination distribution. We instantiated both for metric-induced discrete flow matching and validated them on codec-based zero-shot TTS. Under controlled comparisons, the resulting GibbsTTS achieves the strongest overall performance against the evaluated masked discrete generative baselines.

11 Acknowledgements

This work was supported by JST Moonshot R&D Grant Number JPMJMS2011 and JST SPRING Grant Number JPMJSP2108.

References

- [1] StableTTS. <https://github.com/KdaiP/StableTTS>, 2024.
- [2] P. Anastassiou, J. Chen, J. Chen, Y. Chen, Z. Chen, Z. Chen, J. Cong, L. Deng, C. Ding, L. Gao, M. Gong, P. Huang, Q. Huang, Z. Huang, Y. Huo, D. Jia, C. Li, F. Li, H. Li, J. Li, X. Li, X. Li, L. Liu, S. Liu, S. Liu, X. Liu, Y. Liu, Z. Liu, L. Lu, J. Pan, X. Wang, Y. Wang, Y. Wang, Z. Wei, J. Wu, C. Yao, Y. Yang, Y. Yi, J. Zhang, Q. Zhang, S. Zhang, W. Zhang, Y. Zhang, Z. Zhao, D. Zhong, and X. Zhuang. Seed-tts: A family of high-quality versatile speech generation models. *arXiv preprint arXiv:2406.02430*, 2024.
- [3] Z. Borsos, M. Sharifi, D. Vincent, E. Kharitonov, N. Zeghidour, and M. Tagliasacchi. SoundStorm: Efficient parallel audio generation. *arXiv preprint arXiv:2305.09636*, 2023.
- [4] S. Chen, C. Wang, Z. Chen, Y. Wu, S. Liu, Z. Chen, J. Li, N. Kanda, T. Yoshioka, X. Xiao, J. Wu, L. Zhou, S. Ren, Y. Qian, Y. Qian, J. Wu, M. Zeng, X. Yu, and F. Wei. WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.
- [5] Y. Chen, Z. Niu, Z. Ma, K. Deng, C. Wang, J. Zhao, K. Yu, and X. Chen. F5-TTS: A fairytaler that fakes fluent and faithful speech with flow matching. *arXiv preprint arXiv:2410.06885*, 2024.
- [6] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2022.
- [7] H. Deng, T. Pan, F. Zhang, Y. Liu, Z. Luo, Y. Cui, W. Wang, C. Shen, S. Shan, Z. Zhang, and X. Wang. Uniform discrete diffusion with metric path for video generation. *arXiv preprint arXiv:2510.24717*, 2025.
- [8] Z. Du, C. Gao, Y. Wang, F. Yu, T. Zhao, H. Wang, X. Lv, H. Wang, C. Ni, X. Shi, K. An, G. Yang, Y. Li, Y. Chen, Z. Gao, Q. Chen, Y. Gu, M. Chen, Y. Chen, S. Zhang, W. Wang, and J. Ye. CosyVoice 3: Towards in-the-wild speech generation via scaling-up and post-training. *arXiv preprint arXiv:2505.17589*, 2025.
- [9] Z. Du, Y. Wang, Q. Chen, X. Shi, X. Lv, T. Zhao, Z. Gao, Y. Yang, C. Gao, H. Wang, F. Yu, H. Liu, Z. Sheng, Y. Gu, C. Deng, W. Wang, S. Zhang, Z. Yan, and J. Zhou. CosyVoice 2: Scalable streaming speech synthesis with large language models. *arXiv preprint arXiv:2412.10117*, 2024.
- [10] Z. Gao, S. Zhang, I. McLoughlin, and Z. Yan. Paraformer: Fast and accurate parallel transformer for non-autoregressive end-to-end speech recognition. In *Interspeech*, pages 2063—2067, 2022.
- [11] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Q. Chen, G. Synnaeve, Y. Adi, and Y. Lipman. Discrete flow matching. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [12] H. He, Z. Shang, C. Wang, X. Li, Y. Gu, H. Hua, L. Liu, C. Yang, J. Li, P. Shi, Y. Wang, K. Chen, P. Zhang, and Z. Wu. Emilia: An extensive, multilingual, and diverse speech dataset for large-scale speech generation. In *IEEE Spoken Language Technology Workshop (SLT)*, 2024.
- [13] H. Hu, X. Zhu, T. He, D. Guo, B. Zhang, X. Wang, Z. Guo, Z. Jiang, H. Hao, Z. Guo, X. Zhang, P. Zhang, B. Yang, J. Xu, J. Zhou, and J. Lin. Qwen3-TTS technical report. *arXiv preprint arXiv:2601.15621*, 2026.
- [14] S. ichi Amari. *Information geometry and its applications*, volume 194. Springer, 2016.

- [15] R. Kumar, P. Seetharaman, A. Luebs, I. Kumar, and K. Kumar. High-fidelity audio compression with improved RVQGAN. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [17] R. Luo, X. Xia, L. Wang, L. Chen, R. Shan, J. Luo, M. Yang, and T. Chua. NExT-OMNI: towards any-to-any omnimodal foundation models with discrete flow matching. In *International Conference on Learning Representations (ICLR)*, 2026.
- [18] N.-S. Nguyen, T. V. T. Tran, H.-N. Huynh-Nguyen, T.-S. Hy, and V. Nguyen. DiFlow-TTS: Compact and low-latency zero-shot text-to-speech with factorized discrete flow matching. *arXiv preprint arXiv:2407.05407*, 2024.
- [19] W. Peebles and S. Xie. Scalable diffusion models with transformers. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [20] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever. Robust speech recognition via large-scale weak supervision. In *International Conference on Machine Learning (ICML)*, pages 28492–28518, 2023.
- [21] T. Saeki, D. Xin, W. Nakata, T. Koriyama, S. Takamichi, and H. Saruwatari. UTMOS: utokyo-sarulab system for voicemos challenge 2022. In *Interspeech*, pages 4521–4525, 2022.
- [22] N. Shaul, I. Gat, M. Havasi, D. Severo, A. Sriram, P. Holderrieth, B. Karrer, Y. Lipman, and R. T. Q. Chen. Flow matching with general discrete paths: A kinetic-optimal perspective. In *International Conference on Learning Representations (ICLR)*, 2025.
- [23] N. Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [24] J. Shi, K. Han, Z. Wang, A. Doucet, and M. K. Titsias. Simplified and generalized masked diffusion for discrete data. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2024.
- [25] H. Siuzdak. Vocos: Closing the gap between time-domain and fourier-based neural vocoders for high-quality audio synthesis. In *International Conference on Learning Representations (ICLR)*, 2024.
- [26] J. Su, M. H. M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.
- [27] A. Vasuki and P. Vanathi. A review of vector quantization techniques. *IEEE Potentials*, 25:39–47, 2006.
- [28] C. Wang, S. Chen, Y. Wu, Z. Zhang, L. Zhou, S. Liu, Z. Chen, Y. Liu, H. Wang, J. Li, L. He, S. Zhao, and F. Wei. Neural codec language models are zero-shot text to speech synthesizers. *arXiv preprint arXiv:2301.02111v1*, 2023.
- [29] J. Wang, Y. Lai, A. Li, S. Zhang, J. Sun, N. Kang, C. Wu, Z. Li, and P. Luo. FUDOKI: discrete flow-based unified understanding and generation via kinetic-optimal velocities. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [30] X. Wang, M. Jiang, Z. Ma, Z. Zhang, S. Liu, L. Li, Z. Liang, Q. Zheng, R. Wang, X. Feng, W. Bian, Z. Ye, S. Cheng, R. Yuan, Z. Zhao, X. Zhu, J. Pan, L. Xue, P. Zhu, Y. Chen, Z. Li, X. Chen, L. Xie, Y. Guo, and W. Xue. Spark-TTS: An efficient llm-based text-to-speech model with single-stream decoupled speech tokens. *arXiv preprint arXiv:2503.01710*, 2025.
- [31] Y. Wang, H. Zhan, L. Liu, R. Zeng, H. Guo, J. Zheng, Q. Zhang, X. Zhang, S. Zhang, and Z. Wu. MaskGCT: Zero-shot text-to-speech with masked generative codec transformer. In *International Conference on Learning Representations (ICLR)*, 2025.
- [32] K. Xie, F. Shen, J. Li, F. Xie, X. Tang, and Y. Hu. FireRedTTS-2: Towards long conversational speech generation for podcast and chatbot. *arXiv preprint arXiv:2509.02020*, 2025.

- [33] Y. Xu, J. Cui, F. Cai, Z. Zhu, H. Shang, S. Luan, M. Xu, N. Zhang, Y. Li, J. Cai, and S. Zhu. WAM-Flow: parallel coarse-to-fine motion planning via discrete flow matching for autonomous driving. In *IEEE/CVF International Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [34] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [35] D. Yang, Y. Cai, Y. Saito, L. Wang, and H. Saruwatari. Shallow flow matching for coarse-to-fine text-to-speech synthesis. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2025.
- [36] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 30:495–507, 2022.
- [37] H. Zen, V. Dang, R. Clark, Y. Zhang, R. J. Weiss, Y. Jia, Z. Chen, and Y. Wu. LibriTTS: A corpus derived from LibriSpeech for text-to-speech. In *Proc. Interspeech*, pages 1526–1530, 2019.
- [38] B. Zhang and R. Sennrich. Root mean square layer normalization. In *Annual Conference on Neural Information Processing Systems (NeurIPS)*, 2019.
- [39] L. Zhang and S. Syed. The cosine schedule is fisher-rao-optimal for masked discrete diffusion models. *arXiv preprint arXiv:2508.04884*, 2025.
- [40] S. Zhou, Y. Zhou, Y. He, X. Zhou, J. Wang, W. Deng, and J. Shu. IndexTTS2: A breakthrough in emotionally expressive and duration-controlled auto-regressive zero-shot text-to-speech. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [41] H. Zhu, L. Ye, W. Kang, Z. Yao, L. Guo, F. Kuang, Z. Han, W. Zhuang, L. Lin, and D. Povey. OmniVoice: Towards omnilingual zero-shot text-to-speech with diffusion language models. *arXiv preprint arXiv:2604.00688*, 2026.

A Related works

Codec-based discrete acoustic modeling. Recent TTS systems that model speech with discrete acoustic representations mainly rely on neural audio codecs [36, 6, 15], which convert waveforms into sequences of discrete codec tokens. Most modern neural codec models adopt residual vector quantization (RVQ) [27, 36], where each frame is encoded by a stack of codebooks that progressively refine the quantization residual, so that earlier codebooks capture coarser acoustic information and later codebooks encode finer residual details. VALL-E [28] formulated zero-shot TTS as language modeling over codec tokens, and subsequent autoregressive TTS systems [30, 32, 40, 13] further scaled this paradigm through larger-scale training and improved codec models. Compared with AR codec-based TTS, NAR codec-based generation remains less explored. Current NAR systems generate codec tokens mainly through masked generative modeling formulations [3, 31, 18]. Most of these works treat codec tokens as symbolic discrete labels and rely on a learned embedding lookup, leaving the geometric structure of the codec-token embedding space unexploited at the generative modeling level. In this work, we follow the codec-based TTS model route and build a NAR TTS model on top of the acoustic codec released by MaskGCT, whose encoder follows DAC [15] and decoder follows Vocos [25]. Different from prior works, our method further exploits the geometry of the codec-token latent space through metric-induced discrete flow matching.

Masked generative codec modeling. MaskGCT [31] is a representative codec-based NAR zero-shot TTS model. It first predicts semantic tokens [4] from the input text and then generates acoustic codec tokens conditioned on the predicted semantic tokens. For discrete token modeling, MaskGCT adopts masked generative modeling and performs training and inference in a per-layer strategy over RVQ codebooks. Our model differs from MaskGCT in three aspects. First, we use a direct text-to-codec prediction paradigm without an intermediate semantic-token stage. Second, instead of a mask-source probability path, our model generates codec tokens through metric-induced discrete flow matching, whose intermediate distributions exploit codec-token embedding geometry. Third, we perform full-codebook training and inference, jointly predicting all RVQ codebooks at each generation step. Accordingly, the MaskGCT-style masked discrete diffusion [24] and the standard masked discrete flow matching [11] are included as controlled baselines in our experiments.

Concurrent work: OmniVoice. OmniVoice [41] is a concurrent zero-shot TTS model based on masked discrete diffusion language models. Similar to our work, it adopts a direct text-to-codec paradigm with a full-codebook NAR prediction strategy. The two works, however, have different focuses. OmniVoice targets omnilingual zero-shot TTS through system-level designs. In particular, instead of training a DiT-style backbone with explicit timestep conditioning from scratch, it initializes a bidirectional Transformer from a pre-trained LLM (Qwen3-0.6B [34]). This allows the model to inherit linguistic knowledge and subword tokenization from the pre-trained LLM, avoiding explicit grapheme-to-phoneme conversion and language-specific text normalization. Combined with large-scale multilingual training, these designs lead to strong intelligibility on multilingual benchmarks. Our work instead focuses on the algorithmic aspects of discrete generative modeling. Specifically, the proposed kinetic-optimal scheduler and finite-step moment correction for MI-DFM are orthogonal to system-level choices such as LLM initialization, tokenizer design, frontend simplification, multilingual scaling, and backbone architecture. We therefore view OmniVoice as a complementary concurrent effort, and its engineering designs provide promising directions for future improvement of our system.

B Proofs

Lemma 1 (Scheduler-invariant Fisher-Rao length). *Let $p_t(x) = p(x; \kappa_t)$, where $p(x; \kappa)$ is a differentiable probability path with $\kappa \in [0, \kappa_{\max}]$, and κ_t is a monotonic scheduler with $\kappa_0 = 0$ and $\kappa_1 = \kappa_{\max}$. Then the Fisher-Rao length is invariant to the choice of κ_t :*

$$\ell[\kappa_t] = \int_0^1 \|\dot{p}_t\|_{g_{FR}} dt = \int_0^{\kappa_{\max}} \sqrt{\mathcal{I}(\kappa)} d\kappa \triangleq L, \quad (27)$$

where

$$\mathcal{I}(\kappa) = \sum_x \frac{(\partial_\kappa p(x; \kappa))^2}{p(x; \kappa)} \quad (28)$$

is the Fisher information of the path parameter κ .

Proof. By the chain rule,

$$\dot{p}_t(x) = \partial_\kappa p(x; \kappa_t) \dot{\kappa}_t. \quad (29)$$

Therefore, the Fisher-Rao speed satisfies

$$\|\dot{p}_t\|_{g_{FR}} = \sqrt{\left(\sum_x \frac{(\partial_\kappa p(x; \kappa_t) \dot{\kappa}_t)^2}{p(x; \kappa_t)} \right)} = |\dot{\kappa}_t| \sqrt{\mathcal{I}(\kappa_t)}. \quad (30)$$

Since κ_t is monotonic increasing, $\dot{\kappa}_t \geq 0$, and hence $|\dot{\kappa}_t| = \dot{\kappa}_t$. Thus,

$$\ell[\kappa_t] = \int_0^1 \sqrt{\mathcal{I}(\kappa_t)} \dot{\kappa}_t dt. \quad (31)$$

Using the change of variables $\kappa = \kappa_t$, we obtain

$$\ell[\kappa_t] = \int_{\kappa_0}^{\kappa_1} \sqrt{\mathcal{I}(\kappa)} d\kappa = \int_0^{\kappa_{\max}} \sqrt{\mathcal{I}(\kappa)} d\kappa, \quad (32)$$

which is independent of the specific scheduler κ_t . $\square$

Lemma 2 (Constant-speed characterization). *For a prescribed probability path $p(x; \kappa)$, let $p_t^*(x) = p(x; \kappa_t^*)$ be the path induced by the kinetic-optimal scheduler κ_t^* . Then κ_t^* minimizes $\mathcal{E}_{FR}[\kappa_t]$ over $\kappa_t \in \mathcal{K}$ if and only if it satisfies*

$$\|\dot{p}_t^*\|_{g_{FR}} = L, \quad \forall t \in [0, 1], \quad (33)$$

where L is the scheduler-invariant Fisher-Rao length defined in Lemma 1.

Proof. By the Cauchy-Schwarz inequality,

$$\mathcal{E}_{FR}[\kappa_t] = \int_0^1 \|\dot{p}_t\|_{g_{FR}}^2 dt \geq \left(\int_0^1 \|\dot{p}_t\|_{g_{FR}} dt \right)^2. \quad (34)$$

From Lemma 1, the length term is independent of the scheduler and equals L . Hence,

$$\mathcal{E}_{FR}[\kappa_t] \geq L^2. \quad (35)$$

The equality holds if and only if $\|\dot{p}_t\|_{g_{FR}}$ is constant with respect to t . Since

$$\int_0^1 \|\dot{p}_t\|_{g_{FR}} dt = L, \quad (36)$$

this constant must be L . Therefore, the kinetic-optimal scheduler satisfies $\|\dot{p}_t\|_{g_{FR}} = L$. $\square$

Proposition 1 (Kinetic-optimal time scheduler). *Let $p_t(x) = p(x; \kappa_t)$ be a prescribed path parameterized by a monotonic scheduler κ_t , with $\kappa_0 = 0$ and $\kappa_1 = \kappa_{\max}$. Assume $\mathcal{I}(\kappa) > 0$ on $[0, \kappa_{\max}]$. The kinetic-optimal scheduler and its time derivative are given by*

$$\kappa_t^* = F^{-1}(t), \quad \dot{\kappa}_t^* = \frac{L}{\sqrt{\mathcal{I}(\kappa_t^*)}}, \quad \text{where } F(\kappa) = \frac{\int_0^\kappa \sqrt{\mathcal{I}(\xi)} d\xi}{L}. \quad (37)$$

Proof. By the chain rule,

$$\dot{p}_t(x) = \partial_\kappa p(x; \kappa_t) \dot{\kappa}_t. \quad (38)$$

Substituting this into the Fisher-Rao metric gives

$$\|\dot{p}_t\|_{g_{FR}}^2 = \dot{\kappa}_t^2 \sum_x \frac{(\partial_\kappa p(x; \kappa_t))^2}{p(x; \kappa_t)} = \dot{\kappa}_t^2 \mathcal{I}(\kappa_t). \quad (39)$$

Since $\dot{\kappa}_t \geq 0$,

$$\|\dot{p}_t\|_{g_{FR}} = \dot{\kappa}_t \sqrt{\mathcal{I}(\kappa_t)}. \quad (40)$$

By Lemma 2,

$$\dot{\kappa}_t^* \sqrt{\mathcal{I}(\kappa_t^*)} = L, \quad \text{then } \dot{\kappa}_t^* = \frac{L}{\sqrt{\mathcal{I}(\kappa_t^*)}}. \quad (41)$$

Integrating both sides and applying the change of variable yields

$$\int_0^{\kappa_t^*} \sqrt{\mathcal{I}(\xi)} d\xi = Lt. \quad (42)$$

Taking $t = 1$ gives

$$L = \int_0^{\kappa_{\max}} \sqrt{\mathcal{I}(\xi)} d\xi. \quad (43)$$

Normalizing this accumulated length by the total length L , we define

$$F(\kappa) = \frac{\int_0^\kappa \sqrt{\mathcal{I}(\xi)} d\xi}{\int_0^{\kappa_{\max}} \sqrt{\mathcal{I}(\xi)} d\xi} = \frac{\int_0^\kappa \sqrt{\mathcal{I}(\xi)} d\xi}{L}. \quad (44)$$

Then, we obtain

$$F(\kappa_t^*) = t. \quad (45)$$

Since $\mathcal{I}(\kappa) > 0$ on $[0, \kappa_{\max}]$, F is strictly increasing and hence invertible, then

$$\kappa_t^* = F^{-1}(t). \quad (46)$$

□

C Kinetic-optimal scheduler construction for metric-induced paths

Algorithm 1 determines $\beta_{\max}$ by requiring the endpoint distribution to be sufficiently concentrated on the target token. For each codebook c and target token x_1 , we evaluate the target probability $p_{\beta}^{(c)}(x_1 | x_1)$ under the metric-induced conditional distribution, and define the endpoint concentration score as the minimum target probability over all codebooks and target tokens. Starting from an initial positive value, we double it until the score exceeds $1 - \epsilon$, and then perform binary search on the resulting interval to find the smallest $\beta_{\max}$ satisfying the same criterion. This gives a finite numerical endpoint that approximates the limiting delta distribution up to tolerance ϵ .

Algorithm 2 describes the numerical construction of the kinetic-optimal scheduler. Given the distance matrices $\{\mathbf{D}_c\}_{c=1}^C$, we first evaluate the metric-induced conditional distribution on a finite inverse-temperature grid $\{\beta_i\}_{i=1}^I$. For each grid point, the Fisher information is computed as the variance of the distance to the target token, and then averaged over target tokens and codebooks to obtain a global Fisher information V_i . Using these values, we approximate the cumulative Fisher–Rao arc length by numerical quadrature. The scheduler is then obtained by uniformly redistributing the accumulated arc length over $[0, 1]$ and inverting the resulting ℓ – β relation by linear interpolation. Finally, the derivative table is computed from the constant-speed condition, yielding the lookup tables $\{\beta_j^*\}_{j=1}^T$ and $\{\dot{\beta}_j^*\}_{j=1}^T$.

Algorithm 1 Determine the finite endpoint $\beta_{\max}$

Require: distance matrices $\{\mathbf{D}_c \in \mathbb{R}^{s \times s}\}_{c=1}^C$, endpoint tolerance $\epsilon = 10^{-8}$, initial upper bound $\beta_{\text{init}} = 1$

Ensure: Finite endpoint $\beta_{\max}$

1: Define the metric-induced conditional distribution:

$$p_{\beta}^{(c)}(x | x_1) = \frac{\exp(-\beta \mathbf{D}_c(x, x_1))}{\sum_{y \in [s]} \exp(-\beta \mathbf{D}_c(y, x_1))}, \quad c = 1, \dots, C.$$

2: Define the endpoint concentration score:

$$f(\beta) = \min_{c \in \{1, \dots, C\}} \min_{x_1 \in [s]} p_{\beta}^{(c)}(x_1 | x_1).$$

3: $\beta_{\text{hi}} \leftarrow \beta_{\text{init}}$

4: **while** $f(\beta_{\text{hi}}) < 1 - \epsilon$ **do**

5: $\beta_{\text{hi}} \leftarrow 2\beta_{\text{hi}}$

6: Find the smallest $\beta_{\max} \in [0, \beta_{\text{hi}}]$ by binary search such that

$$f(\beta_{\max}) \geq 1 - \epsilon.$$

7: **return** $\beta_{\max}$

Algorithm 2 Compute the kinetic-optimal scheduler

Require: distance matrices $\{\mathbf{D}_c\}_{c=1}^C$, number of time points $T = 1024$, grid size $I = 4096$, maximum inverse temperature $\beta_{\max}$, numerical constant $\epsilon = 10^{-8}$

Ensure: scheduler table $\{\beta_j^*\}_{j=1}^T$, derivative table $\{\dot{\beta}_j^*\}_{j=1}^T$

1: Construct a uniform inverse-temperature grid: $0 = \beta_1 < \beta_2 < \dots < \beta_I = \beta_{\max}$.

2: **for** $i = 1, \dots, I$ **do**

3: **for** $c = 1, \dots, C$ **do**

4: **for** $x_1 \in [s]$ **do**

5: Compute the metric-induced conditional distribution

$$p_{\beta_i}^{(c)}(x \mid x_1) = \frac{\exp(-\beta_i \mathbf{D}_c(x, x_1))}{\sum_{y \in [s]} \exp(-\beta_i \mathbf{D}_c(y, x_1))}, \quad x \in [s].$$

6: Compute the conditional mean distance

$$\mu_{c,x_1}(\beta_i) = \sum_{x \in [s]} p_{\beta_i}^{(c)}(x \mid x_1) \mathbf{D}_c(x, x_1).$$

7: Compute the conditional distance variance

$$\sigma_{c,x_1}^2(\beta_i) = \sum_{x \in [s]} p_{\beta_i}^{(c)}(x \mid x_1) \mathbf{D}_c(x, x_1)^2 - \mu_{c,x_1}(\beta_i)^2.$$

8: Average over target tokens:

$$V_{c,i} = \frac{1}{s} \sum_{x_1 \in [s]} \sigma_{c,x_1}^2(\beta_i).$$

9: Average over codebooks to obtain the global Fisher information:

$$V_i = \frac{1}{C} \sum_{c=1}^C V_{c,i}.$$

10: Compute the cumulative Fisher–Rao arc length by the trapezoidal rule: $\ell_1 = 0$.

11: **for** $i = 2, \dots, I$ **do**

12:

$$\ell_i = \ell_{i-1} + \frac{\sqrt{\max\{V_i, \epsilon\}} + \sqrt{\max\{V_{i-1}, \epsilon\}}}{2} (\beta_i - \beta_{i-1}).$$

13: Construct uniform time points: $t_j = \frac{j-1}{T-1}$, $j = 1, \dots, T$.

14: **for** $j = 1, \dots, T$ **do**

15: Set the target Fisher–Rao arc length: $\ell_j^* = t_j \ell_I$.

16: Locate the interval: $\ell_{i_j-1} \leq \ell_j^* \leq \ell_{i_j}$, $i_j \in \{2, \dots, I\}$.

17: Linearly interpolate the inverse map between $(\ell_{i_j-1}, \beta_{i_j-1})$ and $(\ell_{i_j}, \beta_{i_j})$:

$$a_j = \frac{\ell_j^* - \ell_{i_j-1}}{\ell_{i_j} - \ell_{i_j-1} + \epsilon}, \quad \beta_j^* = \beta_{i_j-1} + a_j (\beta_{i_j} - \beta_{i_j-1}).$$

18: Interpolate the Fisher information at β_j^* :

$$\tilde{a}_j = \frac{\beta_j^* - \beta_{i_j-1}}{\beta_{i_j} - \beta_{i_j-1} + \epsilon}, \quad V_j^* = V_{i_j-1} + \tilde{a}_j (V_{i_j} - V_{i_j-1}).$$

19: Compute the derivative of the Fisher-uniform scheduler:

$$\dot{\beta}_j^* = \frac{\ell_I}{\sqrt{\max\{V_j^*, \epsilon\}}}.$$

20: **return** $\{\beta_j^*\}_{j=1}^T, \{\dot{\beta}_j^*\}_{j=1}^T$

D Recovering the closed-form kinetic-optimal scheduler for mixture paths

This appendix applies Proposition 1 to the mixture path and recovers its closed-form kinetic-optimal scheduler. This serves as a consistency check of the proposed formulation.

For the mixture $p_t(x)$ defined in Eq 3, we consider it as $p(x | x_1; \kappa) = (1 - \kappa)p(x) + \kappa\delta_{x_1}(x)$.

For a fixed target token x_1 , let $p_1 = p(x_1)$, then

$$p(x_1 | x_1; \kappa) = p_1 + (1 - p_1)\kappa, \quad p(x | x_1; \kappa) = (1 - \kappa)p(x), \quad x \neq x_1, \quad (47)$$

and the Fisher information with respect to κ is

$$\mathcal{I}_{x_1}(\kappa) = \sum_x \frac{(\partial p(x | x_1; \kappa))^2}{p(x | x_1; \kappa)} = \frac{1 - p_1}{(1 - \kappa)(p_1 + (1 - p_1)\kappa)}, \quad (48)$$

and the cumulative Fisher-Rao arc-length $\ell(\kappa)$ is

$$\ell(\kappa) = \int_0^\kappa \sqrt{\mathcal{I}_{x_1}(\xi)} d\xi = \int_0^\kappa \sqrt{\frac{1 - p_1}{(1 - \xi)(p_1 + (1 - p_1)\xi)}} d\xi. \quad (49)$$

Let $y(\xi) = p_1 + (1 - p_1)\xi$, then

$$\ell(\kappa) = \int_{p_1}^{y(\kappa)} \sqrt{\frac{(1 - p_1)^2}{y(1 - y)}} \frac{dy}{1 - p_1} \quad (50)$$

$$= \int_{p_1}^{y(\kappa)} \frac{1}{\sqrt{y(1 - y)}} dy \quad (51)$$

$$= 2 \arcsin(\sqrt{y(\kappa)}) - 2 \arcsin(\sqrt{p_1}) \quad (52)$$

$$= 2 \arcsin(\sqrt{p_1 + (1 - p_1)\kappa}) - 2 \arcsin(\sqrt{p_1}). \quad (53)$$

$$L = \ell(1) = 2 \left(\frac{\pi}{2} - \arcsin(\sqrt{p_1}) \right) = 2 \arccos(\sqrt{p_1}) \quad (54)$$

$$F(\kappa) = \frac{\arcsin(\sqrt{p_1 + (1 - p_1)\kappa}) - \arcsin(\sqrt{p_1})}{\arccos(\sqrt{p_1})} \quad (55)$$

By setting $F(\kappa_t) = t$ and solving for κ_t :

$$\arcsin(\sqrt{p_1 + (1 - p_1)\kappa_t}) = t \arccos(\sqrt{p_1}) + \arcsin(\sqrt{p_1}) \quad (56)$$

Let $\Omega(x_1) = \arccos(\sqrt{p_1})$, then

$$\arcsin(\sqrt{p_1 + (1 - p_1)\kappa_t}) = t\Omega(x_1) + \arcsin(\sqrt{p_1}) = t\Omega(x_1) + \frac{\pi}{2} - \Omega(x_1) \quad (57)$$

Then

$$\sqrt{p_1 + (1 - p_1)\kappa_t} = \cos((1 - t)\Omega(x_1)), \quad (58)$$

$$\kappa_t = \frac{\cos^2((1 - t)\Omega(x_1)) - p_1}{1 - p_1} = 1 + \frac{\cos^2((1 - t)\Omega(x_1)) - 1}{1 - p_1} = 1 - \frac{\sin^2((1 - t)\Omega(x_1))}{\sin^2(\Omega(x_1))}, \quad (59)$$

which is the closed-form kinetic-optimal scheduler as shown in Eq. 11.

E Training and inference of GibbsTTS

Algorithm 3 Training of GibbsTTS

Require: model θ , number of codebooks C , precomputed scheduler table $\{\beta_j^*\}_{j=1}^T$, distance matrices

- $\{\mathbf{D}_c\}_{c=1}^C$
- 1: **repeat**
 - 2: Sample batch (x_1, cond) ▷ cond: text and language ID
 - 3: Sample $t \sim \mathcal{U}[0, 1]$
 - 4: $\beta_t \leftarrow \text{LINEARINTERP}(t, \{\beta_j^*\}_{j=1}^T)$
 - 5: Sample $r \sim \mathcal{U}(0, 0.3)$
 - 6: $m \leftarrow \text{round}(rN)$ ▷ N : valid token length
 - 7: Construct prediction mask ▷ prompt region: 0, target region: 1

$$M_i = \mathbb{1}[i > m].$$
 - 8: **for** $c = 1, \dots, C$ **in parallel do**
 - 9: Sample tokens

$$\tilde{x}_t^{i,c} \sim \text{Categorical}\left(\text{softmax}_{x \in [s]}(-\beta_t \mathbf{D}_c(x, x_1^{i,c}))\right)$$
 ▷ by Gumbel-Max trick.
 - 10: Construct input

$$x_t^{i,c} = \tilde{x}_t^{i,c} \text{ if } M_i = 1 \text{ else } x_1^{i,c}.$$
 - 11: Predict target-token distribution

$$p_{1|t}^\theta = \theta(x_t, t, \text{cond}).$$
 - 12: Define codebook-wise loss weights

$$w_c = 1 - \frac{c-1}{C}.$$
 - 13: Compute loss

$$\mathcal{L}(\theta) = \frac{\sum_{i=1}^N \sum_{c=1}^C M_i w_c \left[-\log p_{1|t}^{\theta, i, c}(x_1^{i,c} \mid x_t, t, \text{cond}) \right]}{\sum_{i=1}^N \sum_{c=1}^C M_i w_c}.$$
 - 14: Update θ via gradient descent on $\mathcal{L}(\theta)$
 - 15: **until** Training end
-

Algorithm 4 Inference of GibbsTTS

Require: model θ , conditioning cond, prompt x_{pr} , target length N , number of steps K , sampling temperature τ , distance matrices $\{\mathbf{D}_c\}_{c=1}^C$, scheduler tables $\{\beta_j^*\}_{j=1}^T$ and $\{\dot{\beta}_j^*\}_{j=1}^T$

1: Construct time grid $t_k = k/K$, $k = 0, \dots, K$.

2: Initialize target tokens

$$x_0^{i,c} \sim \mathcal{U}\{1, \dots, s\}, \quad i = 1, \dots, N, \quad c = 1, \dots, C.$$

3: Set $x_t \leftarrow [x_{\text{pr}}, x_0]$

4: **for** $k = 0, \dots, K - 1$ **do**

5: $t \leftarrow t_k$, $h \leftarrow t_{k+1} - t_k$

6: Interpolate

$$\beta_t \leftarrow \text{LINEARINTERP}(t, \{\beta_j^*\}_{j=1}^T), \quad \beta_{t+h} \leftarrow \text{LINEARINTERP}(t+h, \{\beta_j^*\}_{j=1}^T),$$

$$\dot{\beta}_t \leftarrow \text{LINEARINTERP}(t, \{\dot{\beta}_j^*\}_{j=1}^T).$$

7: **for** $i = 1, \dots, N$ and $c = 1, \dots, C$ **in parallel do**

8: Sample

$$\hat{x}_1^{i,c} \sim p_{1|t}^{\theta,i,c,\tau}(\cdot \mid x_t, t, \text{cond}) \quad \triangleright \text{by Gumbel-Max trick.}$$

9: Define

$$z = x_t^{i,c}, \quad \hat{x}_1 = \hat{x}_1^{i,c}, \quad \text{and for } x \in [s], \text{ let } d_x = D_c(x, \hat{x}_1), \quad d_z = D_c(z, \hat{x}_1).$$

10: Compute

$$p_t(x \mid \hat{x}_1) = \text{softmax}_{x \in [s]}(-\beta_t d_x), \quad u_t(x, z \mid \hat{x}_1) = p_t(x \mid \hat{x}_1) \left[\dot{\beta}_t(d_z - d_x) \right]_+.$$

11: Compute jump intensity and destination distribution

$$\lambda_t(z \mid \hat{x}_1) = \sum_{x \in [s]} u_t(x, z \mid \hat{x}_1), \quad \pi_t(x \mid z, \hat{x}_1) = \frac{u_t(x, z \mid \hat{x}_1)}{\lambda_t(z \mid \hat{x}_1)}.$$

12: Compute the first-order jump probability

$$\rho_{\text{base}} = 1 - \exp(-h\lambda_t(z \mid \hat{x}_1)).$$

13: Compute the reference distribution

$$p_{t+h}(x \mid \hat{x}_1) = \text{softmax}_{x \in [s]}(-\beta_{t+h} d_x).$$

14: Compute

$$A = d_z - \sum_{x \in [s]} p_{t+h}(x \mid \hat{x}_1) d_x, \quad B = \sum_{x \in [s]} \pi_t(x \mid z, \hat{x}_1) (d_z - d_x), \quad \rho^* = \frac{A}{B}.$$

15: Set

$$\rho = \begin{cases} \rho^*, & \text{if } \lambda_t(z \mid \hat{x}_1) > 0, B \neq 0, \text{ and } 0 \leq \rho^* \leq 1, \\ \rho_{\text{base}}, & \text{otherwise.} \end{cases}$$

16: Sample $Z_{\text{jump}} \sim \mathcal{U}[0, 1]$.

17: **if** $Z_{\text{jump}} \leq \rho$ and $\lambda_t(z \mid \hat{x}_1) > 0$ **then**

18: Sample

$$x_{t+h}^{i,c} \sim \pi_t(\cdot \mid z, \hat{x}_1).$$

19: **else**

20: Keep the current token:

$$x_{t+h}^{i,c} \leftarrow z.$$

21: Set $x_t \leftarrow x_{t+h}$.

22: **return** the target part of x_t

F Exact recovery for the mixture path

For the standard mixture path, both the instantaneous rate and the finite-step transition probability admit closed forms. We show that, in this case, the proposed moment correction recovers the exact finite-step transition over $[t, t + h]$, and therefore introduces no additional approximation. This result serves as a consistency check. The correction is mainly useful when the exact finite-step transition is not available in a simple closed form, as in the metric-induced paths considered in Section 4.

We work with the standard mixture path

$$p_t(x \mid x_0, x_1) = (1 - \kappa_t)\delta_{x_0}(x) + \kappa_t\delta_{x_1}(x),$$

where κ_t is monotone increasing and $x_0 \neq x_1$. In this case, the CTMC has only one possible non-trivial transition, from x_0 to x_1 , with instantaneous rate

$$\lambda_t = \frac{\dot{\kappa}_t}{1 - \kappa_t}.$$

The exact probability of jumping from x_0 to x_1 over the finite interval $[t, t + h]$, conditioned on $X_t = x_0$, is

$$\rho_{\text{exact}} = 1 - \exp\left(-\int_t^{t+h} \frac{\dot{\kappa}_s}{1 - \kappa_s} ds\right) = \frac{\kappa_{t+h} - \kappa_t}{1 - \kappa_t}.$$

The standard first-order solver freezes the instantaneous rate at time t , giving

$$\rho_{\text{base}} = 1 - \exp(-h\lambda_t) = 1 - \exp\left(-\frac{h\dot{\kappa}_t}{1 - \kappa_t}\right),$$

which agrees with ρ_{exact} only in the limit $h \rightarrow 0$.

We now show that the proposed moment correction recovers ρ_{exact} . For the mixture path, the jump destination distribution is deterministic:

$$\pi_t(\cdot \mid x_0, x_1) = \delta_{x_1}.$$

We choose the scalar moment as the target-state indicator,

$$\phi_t(x \mid x_1) = \mathbb{I}\{x = x_1\}.$$

When the current state is x_0 , we have

$$\phi_t(x_0 \mid x_1) = 0, \quad \bar{\phi}_t(x_0, x_1) = \mathbb{E}_{y \sim \pi_t(\cdot \mid x_0, x_1)}[\phi_t(y \mid x_1)] = 1.$$

We take the reference moment to be the state-conditional finite-step moment, namely the probability of being at x_1 at time $t + h$ given $X_t = x_0$:

$$m_{t+h}(x_0, x_1) = \mathbb{P}(X_{t+h} = x_1 \mid X_t = x_0) = \frac{\kappa_{t+h} - \kappa_t}{1 - \kappa_t}.$$

Substituting these quantities into the generic moment-correction formula in Eq. 21 gives

$$\rho^* = \frac{m_{t+h}(x_0, x_1) - \phi_t(x_0 \mid x_1)}{\bar{\phi}_t(x_0, x_1) - \phi_t(x_0 \mid x_1)} = \frac{\kappa_{t+h} - \kappa_t}{1 - \kappa_t} = \rho_{\text{exact}}.$$

Thus, for the standard mixture path, the correction recovers the known exact finite-step transition. Since this transition is already available in closed form, the correction is redundant in this special case and mainly serves as a consistency check.

For metric-induced paths, however, the corresponding state-conditional finite-step moment is generally not available in closed form, since it would require solving the finite-step transition kernel of a time-inhomogeneous CTMC. Section 4 therefore uses the unconditional expectation under p_{t+h} as a practical reference moment, which makes the metric-induced correction a finite-step approximation that tracks a chosen scalar moment, rather than an exact CTMC transition.

G Hyperparameter search of β

As introduced in Section 2.3, for the original MI-DFM method proposed in [22], we need to select well-performing a and c for

$$\beta_t = c\left(\frac{t}{1-t}\right)^a.$$

To reduce training costs, we conduct hyperparameter search on the Base variants and select the hyperparameters according to their performance on the validation set. Although we explored a wider range of candidate values, Table 6 reports only the values around the finally selected hyperparameter for clarity.

Table 6: Hyperparameter search results for the MI-DFM on the validation set. The best value for each metric is highlighted in **bold**.

a	c	UTMOS $\uparrow$	WER (%) $\downarrow$	SIM $\uparrow$
1	1	3.688	3.190	0.685
2.5	1	3.973	3.139	0.704
5	1	3.995	3.200	0.720
7.5	1	3.859	3.397	0.705
10	1	4.002	3.442	0.718
20	1	3.882	4.345	0.699
5	0.5	3.522	24.594	0.670
5	1.5	3.903	3.074	0.687

Following the evaluation protocol described in Section 6.3, we select $a = 5$ and $c = 1$ for the Large variant experiments.

H Shared and per-codebook schedulers

Since the proposed kinetic-optimal scheduler for MI-DFM does not require hyperparameter search, a natural question is whether each RVQ codebook should use an individual scheduler. We therefore compare a shared scheduler with per-codebook schedulers on the GibbsTTS-Base variant.

Table 7: Ablation study on shared and per-codebook kinetic-optimal schedulers. The best value for each metric is highlighted in **bold**.

Method	Seed-TTS test-en			CosyVoice 3 en		
	UTMOS $\uparrow$	WER (%) $\downarrow$	SIM $\uparrow$	UTMOS $\uparrow$	WER (%) $\downarrow$	SIM $\uparrow$
Shared scheduler	3.631	1.961	0.711	3.120	7.224	0.649
Per-codebook scheduler	3.657	1.902	0.705	3.072	7.661	0.647

As shown in Table 7, the per-codebook scheduler does not bring consistent improvements over the shared scheduler. Although it slightly improves UTMOS and WER on Seed-TTS test-en, the shared scheduler performs better in speaker similarity and achieves better results on all metrics of CosyVoice 3 en.

Moreover, the per-codebook scheduler requires storing separate β and $\dot{\beta}$ lookup tables for different RVQ codebooks, introducing additional memory overhead. Therefore, we use a shared kinetic-optimal scheduler in our final model.

I Training objective

As discussed in Section 5, we adopt full-codebook training and inference, where all RVQ codebooks are predicted jointly. We further introduce codebook-wise loss weights in the training objective.

We first discuss the motivation for full-codebook training and inference. As mentioned in Appendix A, the concurrent work OmniVoice [41] adopts a similar full-codebook prediction strategy. However, in our setting, where the model is trained from scratch with a relatively small number of parameters, treating all RVQ codebooks equally makes optimization unstable in the early stage of training. In particular, we observed gradient explosion when no codebook-wise weighting was used. Since earlier RVQ codebooks capture coarser acoustic information, while later codebooks mainly encode residual details with progressively reduced perceptual importance, we apply linearly decayed codebook-wise weights, as shown in Lines 12–13 of Algorithm 3. This reduces the contribution of later codebooks and stabilizes training, allowing the model to converge reliably.

We also compare our full-codebook strategy with the per-layer training and inference strategy used in MaskGCT. The results are shown in Table 8.

Table 8: Ablation study on full-codebook and per-layer training/inference strategies. The best value for each metric is highlighted in **bold**.

Method	NFE	Seed-TTS test-en			CosyVoice 3 en		
		UTMOS↑	WER (%) ↓	SIM ↑	UTMOS↑	WER (%) ↓	SIM ↑
Full-codebook	32	3.631	1.961	0.711	3.120	7.224	0.649
Per-layer	32	3.648	4.048	0.672	3.048	14.366	0.593
	64	3.745	3.218	0.674	3.233	16.769	0.594
	66	3.751	3.377	0.675	3.226	16.988	0.592
	82	3.357	4.098	0.650	2.727	15.677	0.572
	192	3.536	4.056	0.663	2.955	15.636	0.588

Since the codec that we use contains 12 RVQ codebooks, the total NFE in the per-layer strategy is allocated across the codebooks. Following MaskGCT, we consider the allocation

$$[40, 16, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1],$$

which results in 66 total steps. We also evaluate several other allocations:

$$\begin{aligned} 32 \text{ steps} &: [16, 6, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], \\ 64 \text{ steps} &: [40, 14, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1], \\ 82 \text{ steps} &: [16, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6], \\ 192 \text{ steps} &: [16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16, 16]. \end{aligned}$$

The results show that, under our model and experimental setting, the per-layer strategy used in MaskGCT generally performs worse than our full-codebook strategy. Although per-layer inference can allocate more refinement steps to earlier codebooks, it is less efficient when the total NFE is fixed, since the steps are distributed across RVQ layers instead of jointly updating all codebooks at each step. In contrast, full-codebook inference updates all codebooks simultaneously, which leads to better intelligibility and speaker similarity in our experiments.

In addition, we initially attempted to include the autoregressive (AR) model as a baseline. However, even with codebook-wise loss weighting, the AR model failed to learn stable full-codebook prediction. The training loss did not decrease effectively, and the generated samples did not form normal speech. This observation suggests that, in our setting, NAR modeling is more suitable for full-codebook prediction across all RVQ codebooks.

J Duration predictor

Although MaskGCT describes a flow-matching-based duration predictor in the paper, its released implementation uses a simple rule-based duration estimation:

$$L(\text{target speech}) = \frac{L(\text{prompt speech})}{L(\text{prompt phoneme})} L(\text{target phoneme}), \quad (60)$$

where $L(\cdot)$ denotes the sequence length. This rule assumes that the speech-to-phoneme length ratio is approximately consistent between the prompt and the target utterance.

We found that this heuristic works reasonably well in standard cases, but it can produce unreliable duration estimates when the prompt speaking rate is out of distribution. To improve robustness, we clip the prompt-derived ratio using the average ratio computed from the training set:

$$r_{\text{prompt}} = \frac{L(\text{prompt speech})}{L(\text{prompt phoneme})}, \quad r = \text{clip} \left(r_{\text{prompt}}, \gamma \bar{r}, \frac{\bar{r}}{\gamma} \right), \quad (61)$$

$$L(\text{target speech}) = r L(\text{target phoneme}), \quad (62)$$

where $\bar{r}$ is the average speech-to-phoneme length ratio in the training set, and $\gamma \in (0, 1]$ controls the strength of the clipping. A smaller γ gives a wider valid range and therefore behaves closer to the original rule, while $\gamma = 1$ forces all utterances to use the global average ratio $\bar{r}$.

In our training set, the average ratios $\bar{r}$ are 3.224 and 3.286 for English and Chinese, respectively. We evaluate the effect of γ using GibbsTTS-Base on the validation set. To focus on challenging cases, we rank the utterances in the validation set by their prompt speech-to-phoneme ratio deviation and select the top 10% most out-of-distribution utterances. The results are shown in Table 9.

Table 9: Effect of duration-ratio clipping on speaking-rate outliers. The utterances are derived from the validation set. The best value for each metric is highlighted in **bold**.

γ	UTMOS $\uparrow$	WER (%) $\downarrow$	SIM $\uparrow$
No clipping	3.992	5.015	0.686
0.2	3.992	5.015	0.686
0.4	3.992	5.015	0.686
0.6	3.998	4.523	0.688
0.8	4.017	4.130	0.698
1.0	4.048	3.835	0.686

When $\gamma \leq 0.4$, the results are identical to those without clipping, indicating that the clipping range is still too wide to affect the selected utterances. As γ increases, duration estimation becomes more constrained and the WER consistently improves. Although $\gamma = 1.0$ achieves the best UTMOS and WER in this subset, it makes the duration estimate entirely determined by the global average ratio $\bar{r}$. This removes prompt-dependent speaking-rate variation and leads to an overly deterministic duration predictor, which may be undesirable for broader application scenarios. Therefore, we fix $\gamma = 0.8$ in all experiments as a practical trade-off between robustness and prompt-dependent duration modeling.

K Inference sampling temperature selection

During inference, the logits predicted by the model are divided by a sampling temperature before token sampling. We select this temperature on the validation set. In practice, we observe that the suitable temperature range differs between MI-DFM models and masked DFM or discrete diffusion models. Here, we use GibbsTTS-Large as an example to illustrate the temperature-search pipeline.

Table 10: Temperature search results for GibbsTTS-Large on the validation set. The best value for each metric is highlighted in **bold**.

Temperature	UTMOS $\uparrow$	WER (%) $\downarrow$	SIM $\uparrow$
0.2	4.039	3.215	0.749
0.4	4.079	3.013	0.751
0.6	4.084	3.033	0.752
0.8	4.048	3.053	0.748
1.0	3.860	3.250	0.737

As shown in Table 10, temperature 0.6 achieves the best UTMOS and SIM while maintaining a WER close to the best value. Following the evaluation protocol described in Section 6.3, we use 0.6 as the sampling temperature for GibbsTTS. For MI-DFM-based models, the selected temperature is consistently 0.6 in our experiments. Masked generative baselines use lower temperatures: 0.1 for masked DFM with the closed-form kinetic-optimal scheduler, and 0.2 for the remaining models.

L Effect of number of function evaluations

We further study the effect of the number of function evaluations (NFE) during inference. Besides the main 32 NFE setting, Tables 11 and 12 report additional results with 16 and 64 NFE. For MI-DFM, the proposed moment corrector improves performance over the corresponding solver without the corrector in nearly all settings, particularly on UTMOS and WER/CER metrics. The improvement is especially clear in intelligibility metrics at low and moderate NFE, where finite-step discretization errors are more significant. On UTMOS, the corrector provides consistent gains across all NFE settings. With the numerical kinetic-optimal scheduler, MI-DFM with the corrector achieves the best UTMOS on both Seed-TTS and CosyVoice 3 test sets, showing that the proposed scheduler and corrector are effective for improving naturalness.

Compared with grid-searched heuristic schedulers, the numerical kinetic-optimal scheduler achieves competitive or better overall performance without downstream scheduler search. Although grid-searched schedulers can occasionally obtain slightly better intelligibility metrics, the numerical scheduler provides stronger naturalness and similarity in most settings while avoiding manual scheduler tuning.

Masked DFM also benefits from increasing NFE, but its performance depends strongly on the scheduler. In particular, the closed-form kinetic-optimal scheduler provides strong results for masked DFM, whereas MaskGCT-style schedules require larger NFE to approach competitive performance. Masked DD is more sensitive to the scheduler. Although the closed-form KO scheduler improves naturalness, it leads to much worse WER/CER, suggesting that high perceptual quality does not necessarily imply good intelligibility for this decoding formulation.

Table 11: Objective evaluation results on Seed-TTS test sets. The best value for each metric is highlighted in **bold**.

Method	Scheduler	NFE	test-en			test-zh		
			UTMOS↑	WER (%)↓	SIM↑	UTMOS↑	CER (%)↓	SIM↑
Ground truth	—	—	3.527	2.020	0.734	2.782	1.327	0.755
Codec reconstructed	—	—	3.407	2.229	0.695	2.564	1.472	0.725
MI-DFM (GibbsTTS)	Numerical KO	16	3.548	2.037	0.736	2.594	1.569	0.785
		32	3.651	1.777	0.743	2.712	1.327	0.790
		64	3.704	1.785	0.742	2.779	1.223	0.790
MI-DFM w/o corrector	Numerical KO	16	3.288	2.363	0.715	2.323	2.364	0.769
		32	3.403	2.120	0.723	2.447	1.777	0.775
		64	3.450	2.037	0.727	2.490	1.562	0.777
MI-DFM	Grid-searched	16	3.474	1.869	0.721	2.477	1.519	0.774
		32	3.617	1.793	0.729	2.628	1.297	0.784
		64	3.683	1.760	0.730	2.704	1.349	0.785
MI-DFM w/o corrector	Grid-searched	16	3.245	2.112	0.706	2.242	2.002	0.759
		32	3.380	2.070	0.711	2.381	1.637	0.767
		64	3.421	1.928	0.717	2.442	1.580	0.772
Masked DFM	Closed-form KO	16	3.533	2.296	0.739	2.519	2.650	0.782
		32	3.639	1.969	0.742	2.656	1.536	0.788
		64	3.661	1.793	0.743	2.687	1.342	0.791
Masked DFM	DiFlow-TTS	16	3.471	1.869	0.727	2.453	1.507	0.781
		32	3.546	1.827	0.728	2.559	1.308	0.785
		64	3.588	1.810	0.728	2.592	1.195	0.785
Masked DFM	MaskGCT	16	2.800	5.272	0.673	1.787	9.419	0.713
		32	3.269	2.724	0.712	2.195	3.140	0.762
		64	3.449	2.405	0.724	2.417	1.616	0.776
Masked DD	Closed-form KO	16	3.469	9.127	0.720	2.507	10.474	0.779
		32	3.634	5.808	0.731	2.706	6.033	0.787
		64	3.682	4.802	0.732	2.773	4.599	0.789
Masked DD	DiFlow-TTS	16	2.191	28.671	0.587	1.557	33.345	0.644
		32	2.768	9.303	0.672	1.825	10.711	0.734
		64	3.052	4.861	0.697	2.072	3.800	0.763
Masked DD	MaskGCT	16	3.306	2.615	0.715	2.273	1.999	0.774
		32	3.415	2.338	0.721	2.387	1.583	0.776
		64	3.457	2.137	0.722	2.442	1.502	0.778

Table 12: Objective evaluation results on CosyVoice 3 test sets. The best value for each metric is highlighted in **bold**.

Method	Scheduler	NFE	en			zh		
			UTMOS↑	WER (%)↓	SIM↑	UTMOS↑	CER (%)↓	SIM↑
MI-DFM (GibbsTTS)	Numerical KO	16	3.042	4.711	0.686	2.304	5.546	0.781
		32	3.238	4.110	0.691	2.438	4.144	0.780
		64	3.312	4.206	0.677	2.486	4.069	0.768
MI-DFM w/o corrector	Numerical KO	16	2.623	6.090	0.662	1.961	7.345	0.765
		32	2.850	4.616	0.668	2.135	5.485	0.772
		64	2.938	4.302	0.676	2.173	5.047	0.777
MI-DFM	Grid-searched	16	2.773	4.192	0.666	2.036	4.165	0.767
		32	3.009	4.506	0.674	2.189	3.706	0.772
		64	3.132	4.411	0.669	2.279	3.768	0.765
MI-DFM w/o corrector	Grid-searched	16	2.390	5.107	0.642	1.806	5.266	0.744
		32	2.616	4.547	0.653	1.939	4.274	0.755
		64	2.690	4.916	0.659	1.982	4.007	0.759
Masked DFM	Closed-form KO	16	2.819	6.923	0.681	2.120	8.377	0.769
		32	3.049	5.162	0.695	2.294	4.855	0.781
		64	3.098	4.342	0.692	2.332	4.247	0.782
Masked DFM	DiFlow-TTS	16	2.783	4.302	0.665	2.014	4.117	0.770
		32	2.925	4.288	0.673	2.141	3.727	0.777
		64	2.970	3.960	0.670	2.182	3.494	0.776
Masked DFM	MaskGCT	16	1.911	21.330	0.533	1.523	16.973	0.611
		32	2.354	8.767	0.614	1.789	7.235	0.698
		64	2.658	5.216	0.653	1.978	5.355	0.753
Masked DD	Closed-form KO	16	2.730	26.751	0.662	2.192	21.432	0.763
		32	3.042	18.353	0.677	2.401	14.156	0.776
		64	3.144	14.939	0.681	2.476	11.735	0.781
Masked DD	DiFlow-TTS	16	1.535	100.451	0.432	1.369	57.556	0.523
		32	1.885	36.133	0.562	1.494	29.180	0.673
		64	2.201	13.328	0.624	1.622	11.742	0.735
Masked DD	MaskGCT	16	2.466	7.702	0.649	1.812	6.052	0.755
		32	2.657	6.719	0.655	1.903	4.575	0.762
		64	2.725	5.189	0.657	1.964	4.623	0.766